

中国科学院大学
University of Chinese Academy of Sciences

Generative Models & AI4S Applications

Lecture 10 Generative Adversarial Networks II

Fall 2026

Liu Yunfan (刘云帆)

liuyunfan@ucas.ac.cn

Announcements

- Assignment 1 due on **Oct 25th**

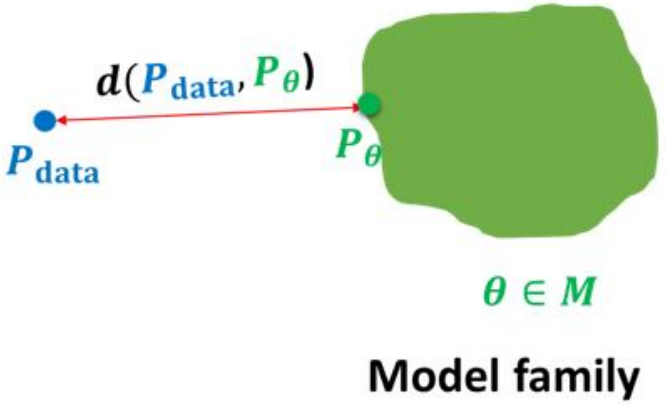
- Assignment 2 will be released recently

Recap

Images of dogs



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$

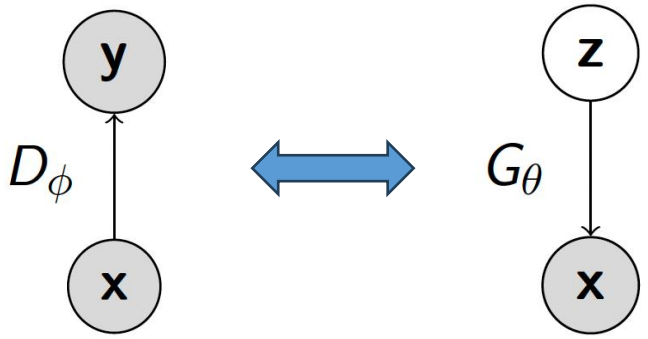


Dog Image Generator

- Generative models learned so far (before GAN):
  - **Autoregressive Models:** $p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i \mid x_{<i})$
  - **Variational Autoencoders:** $p_\theta(x) = \int p_\theta(x, z)\, dz$
  - **Normalizing Flow Models:** $p_X(\boldsymbol{x}; \theta) = p_Z\left(\boldsymbol{f}_\theta^{-1}(\boldsymbol{x})\right)\left|\det\left(\frac{\partial \boldsymbol{f}_\theta^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}}\right)\right|$
- Learning objective: **Maximum Likelihood / Minimize** $\boldsymbol{D_{KL}(p_\theta \,||\, p_{\text{data}})}$

Recap

- **Towards Likelihood-Free Training**
  - **GAN:** A **two-player minimax game** between generator and discriminator



  - **Discriminator**
    - ✓ Any function tries to **classify** "real" data and "fake" generations
    - ✓ $D_\phi(x) = P_\phi(y = 1 \mid x)$
  - **Generator**
    - ✓ Directed, **latent variable model** with **deterministic mapping** $G_\theta: z \to x$
    - ✓ Improve generation quality with the guidance provided by D_ϕ

Recap

- **Training Objectives**

$$\min_G \max_D V(G, D) = E_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + E_{\mathbf{x}\sim p_G}[\log(1 - D(\mathbf{x}))]$$

➢ Optimal discriminator D_G^*

$$D_\theta^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_\theta(\mathbf{x})}$$

➔ **With D_G^***, the generator minimizes a scaled and shifted **JS Divergence**

$$\min_G 2D_{JSD}[p_{\text{data}}, p_G] - \log 4$$

$$D_{JSD}[p, q] = \frac{1}{2}\left(D_{KL}\left[p, \frac{p+q}{2}\right] + D_{KL}\left[q, \frac{p+q}{2}\right]\right)$$

Recap

- **Issues of GAN Training**
  - **Mode Collapse:** "Mode-seeking" nature of Jenson-Shannon Divergence
  - **Vanishing Gradient:** Over-confident D in early training stage
  - Solutions are mostly empirical (**"GAN-hacks"**)
    - ✓ Alternate network capabilities
    - ✓ Label smoothing
    - ✓ Adding noise to data
    - ✓ " How to Train a GAN? Tips and tricks to make GANs work "
    - ✓ …
- **Question:** Any **theoretical or systematical improvement** of GAN?



Lecture Outline

- **Wasserstein GAN (WGAN)**
- **Adversary as a Loss Function**
- **“Scaling” of GAN Models**

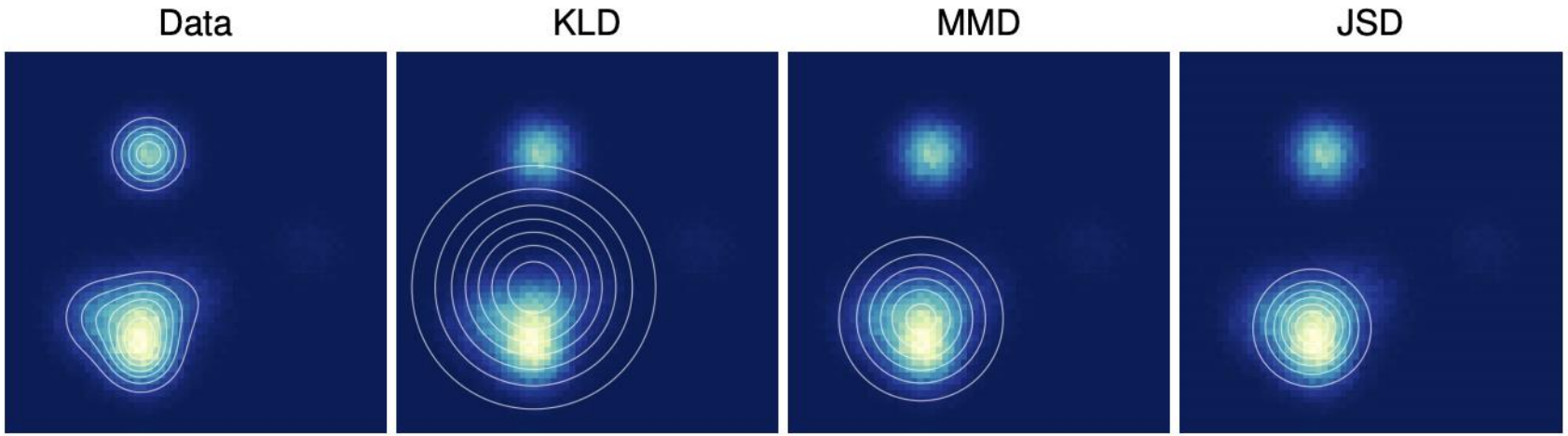


Lecture Outline

- **Wasserstein GAN (WGAN)**
- **Adversary as a Loss Function**
- **“Scaling” of GAN Models**

Wasserstein GAN (WGAN)

- **Recap on JS Divergence**
  - **Note:** One metric for measuring $d(p_{\text{data}}, p_\theta)$



  - With D_G^*, the generator minimizes the JS Divergence (looks nice!)

$$\min_G 2D_{JSD}[p_{\text{data}}, p_G] - \log 4$$

  - **Question:** What are the limitations of JS Divergence?

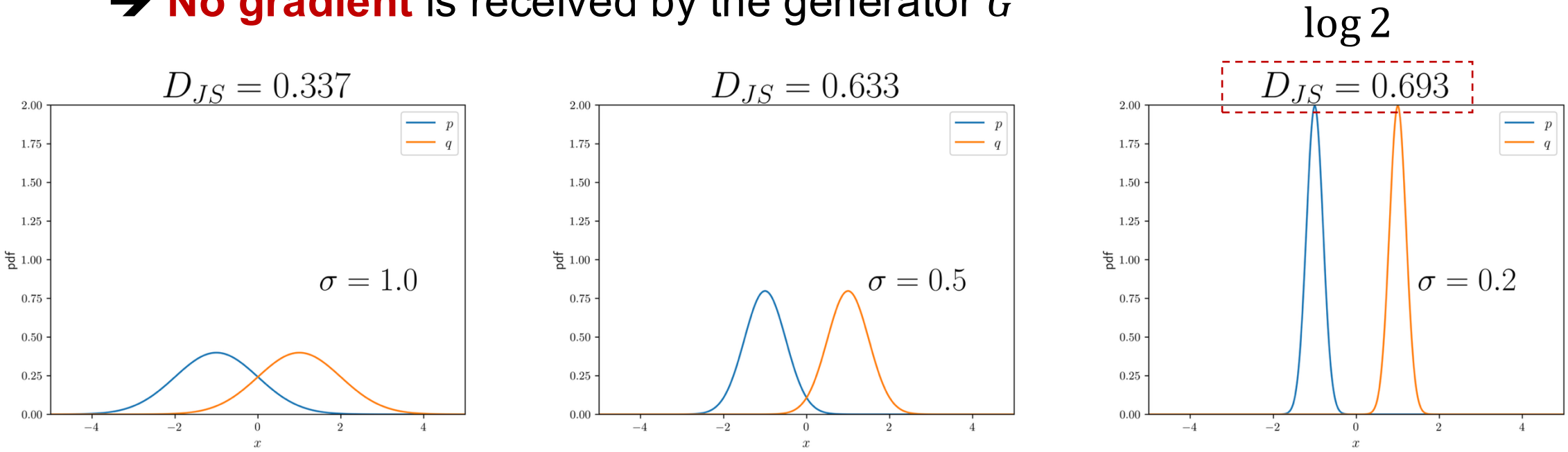
Wasserstein GAN (WGAN)

- **Limitations of JS Divergence**

$$D_{JSD}[p,q] = \frac{1}{2}\left(D_{KL}\left[p, \frac{p+q}{2}\right] + D_{KL}\left[q, \frac{p+q}{2}\right]\right)$$

➢ **Note:** If p and q do not overlap, then D_{JSD} is a **constant** ($\log 2$)

➔ **No gradient** is received by the generator G



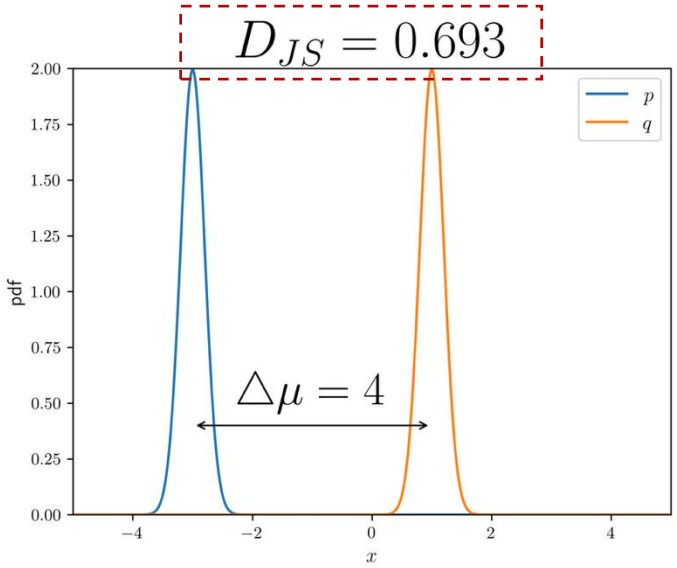
Wasserstein GAN (WGAN)

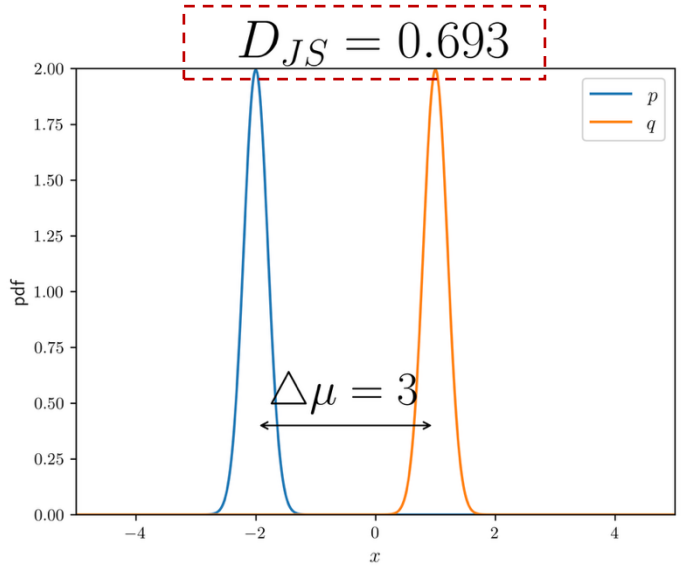
- **Limitations of JS Divergence**

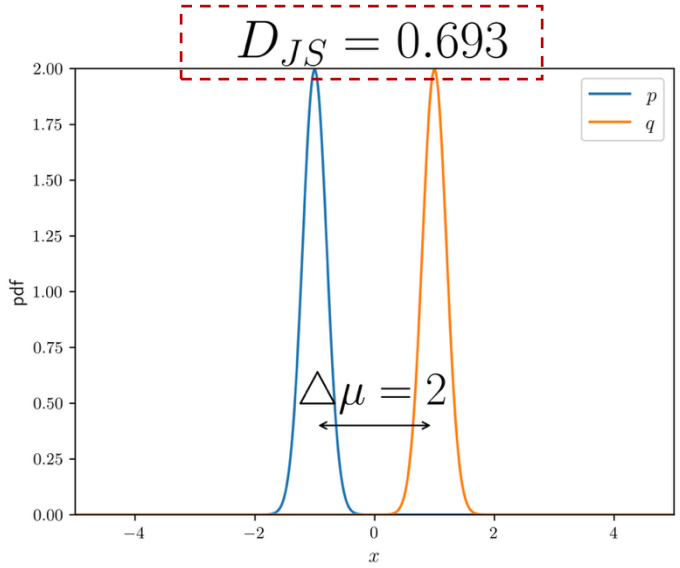
$$D_{JSD}[p,q] = \frac{1}{2}\left(D_{KL}\left[p, \frac{p+q}{2}\right] + D_{KL}\left[q, \frac{p+q}{2}\right]\right)$$

➢ **Note:** If p and q do not overlap, then D_{JSD} is a **constant** ($\log 2$)

➔ **No gradient** is received by the generator G

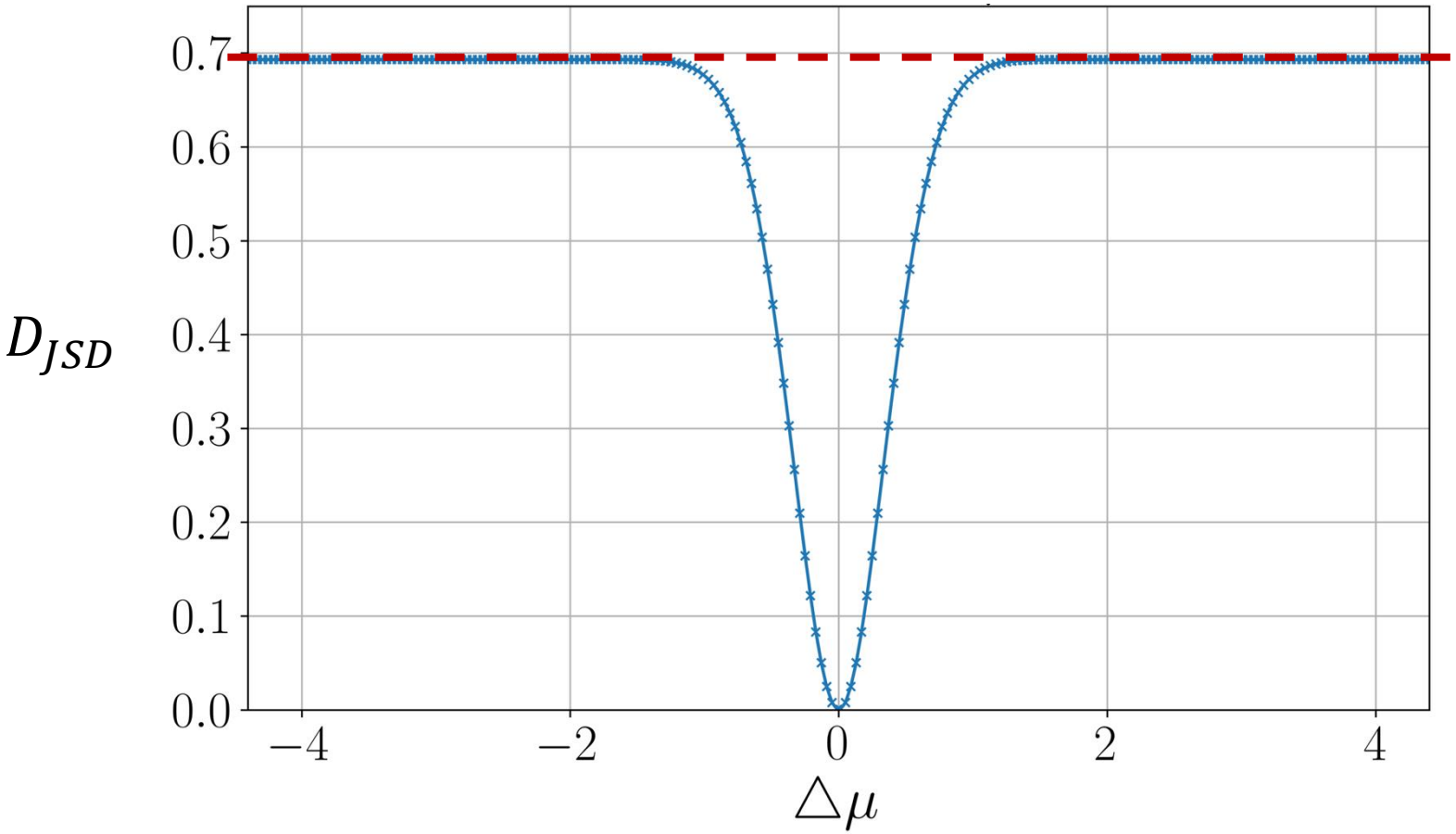




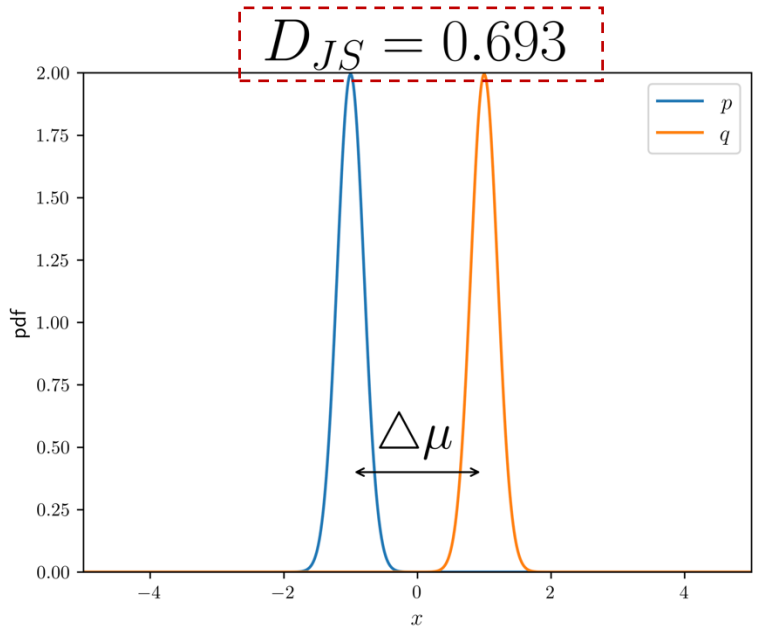


Wasserstein GAN (WGAN)

- **Limitations of JS Divergence**
  - **Note:** D_{JSD} is informative only if p and q are close

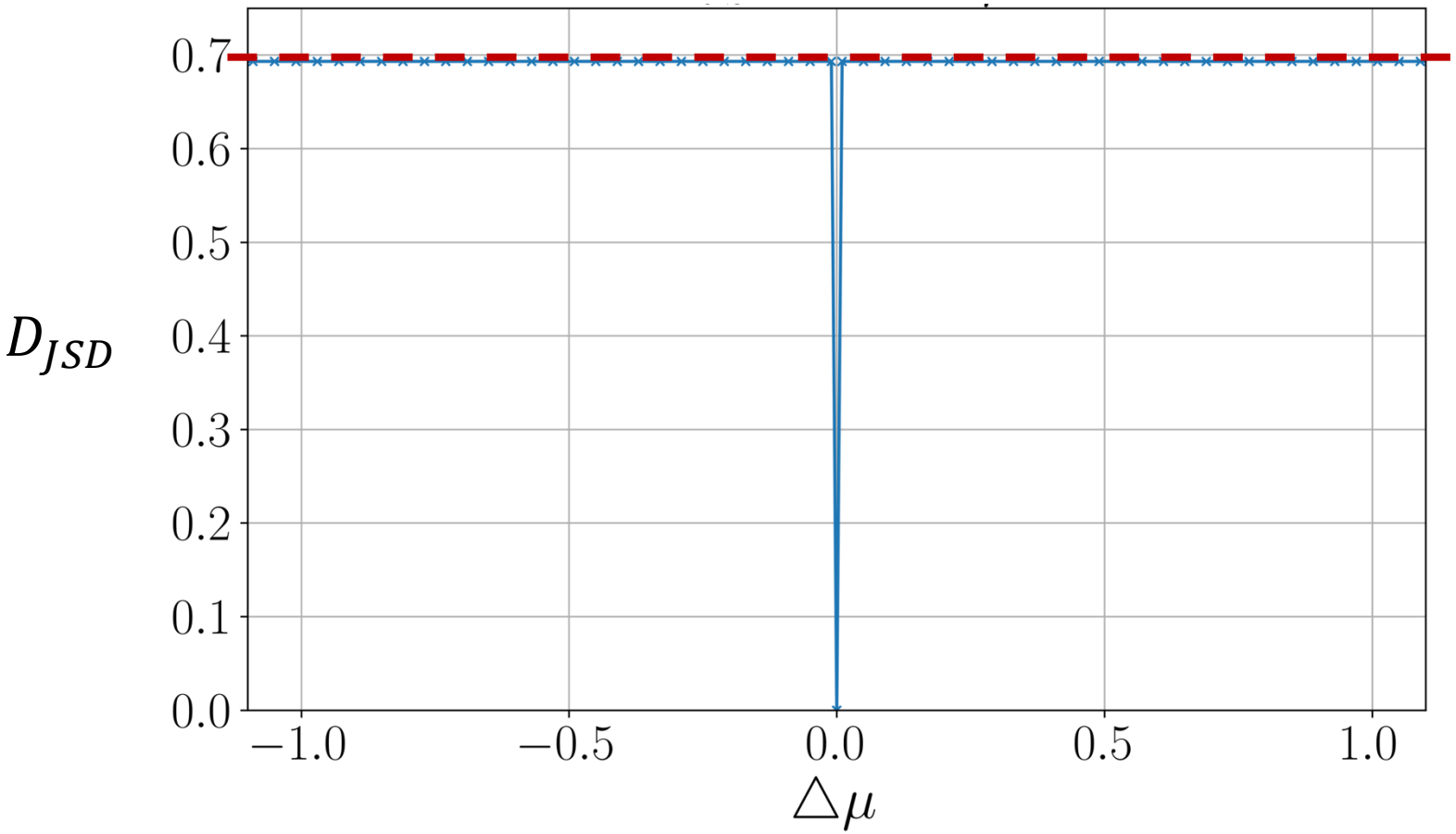


D_{JSD} is **saturated**
(no gradient to generator)

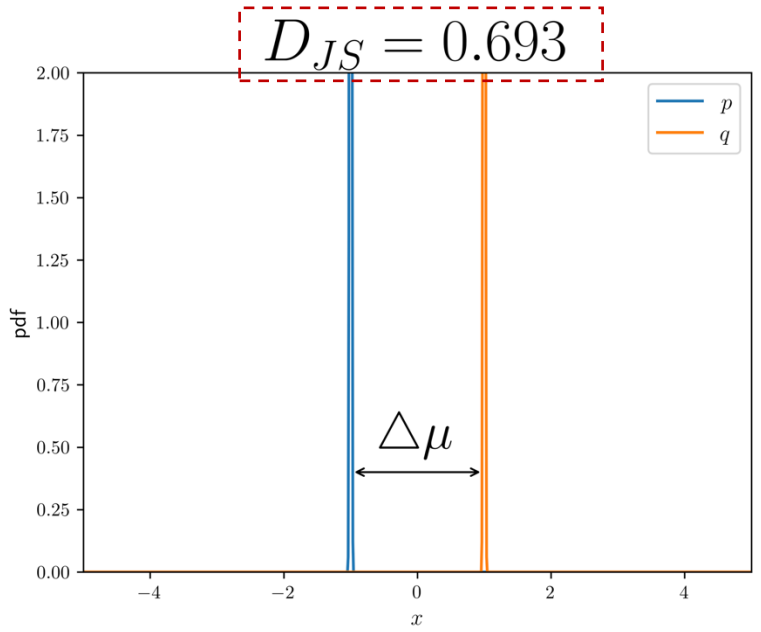


Wasserstein GAN (WGAN)

- **Limitations of JS Divergence**
  - **Extreme Case:** D_{JSD} completely fails to measure $d(p, q)$

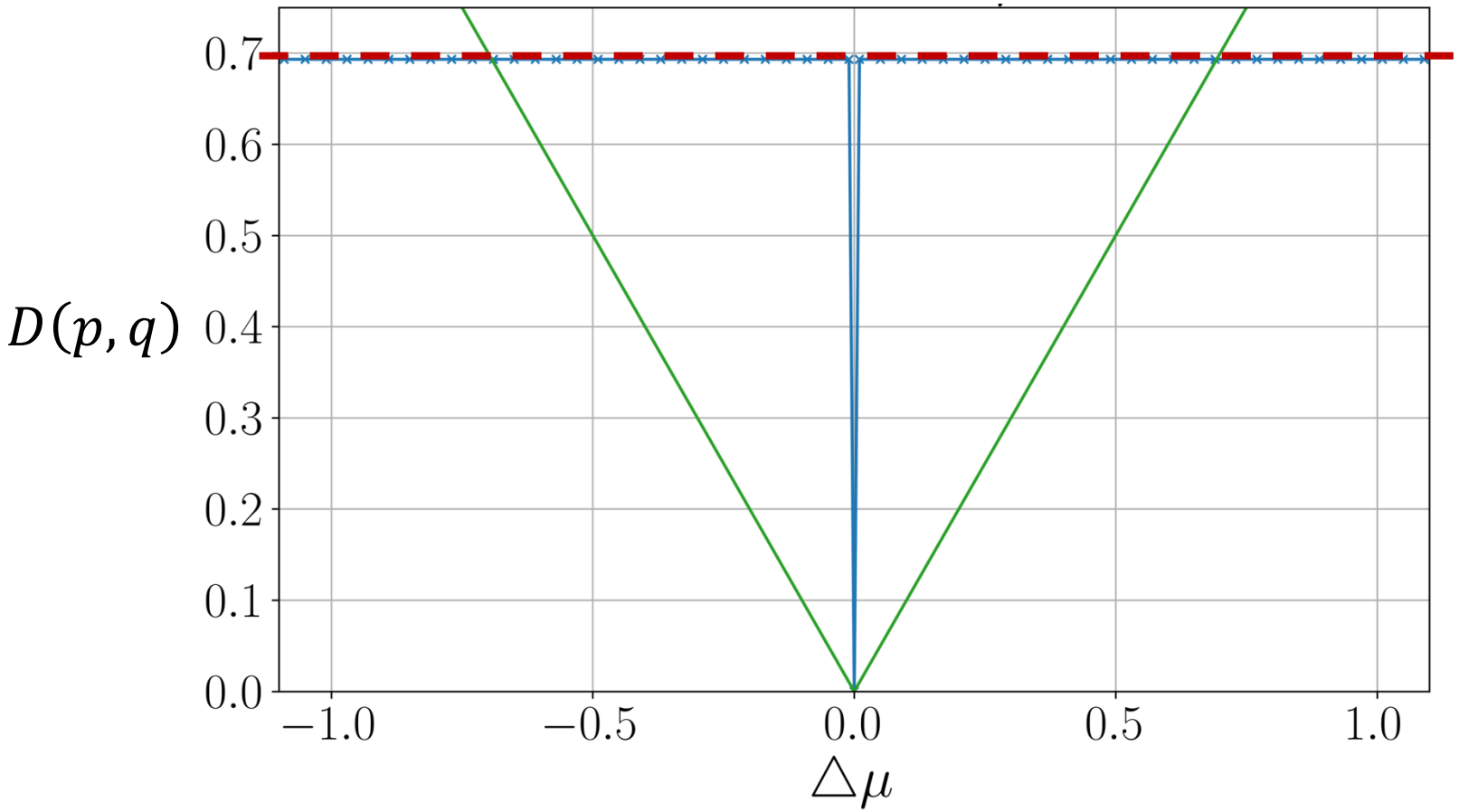


D_{JSD} saturates as long as $p \neq q$ (no distance info)

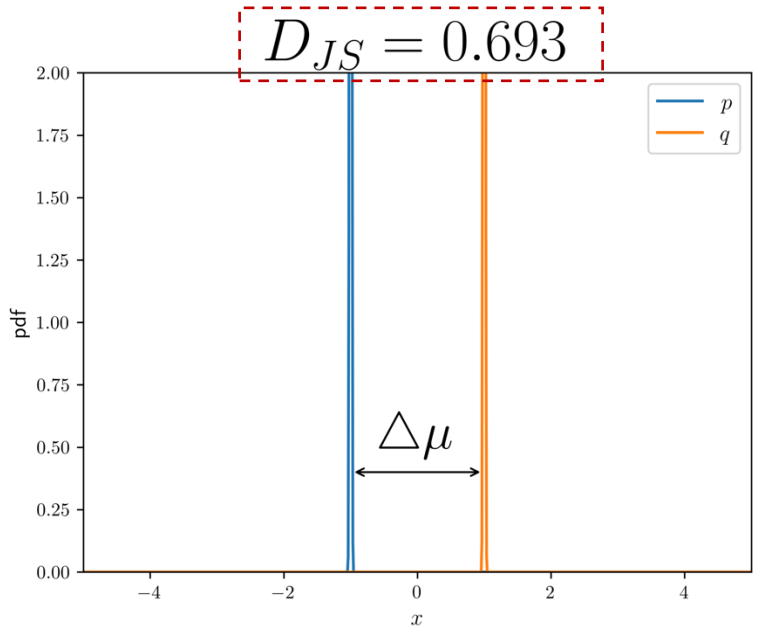


Wasserstein GAN (WGAN)

- **Limitations of JS Divergence**
  - **Goal:** A **"smoother" distance** defined when p and q have **disjoint support**

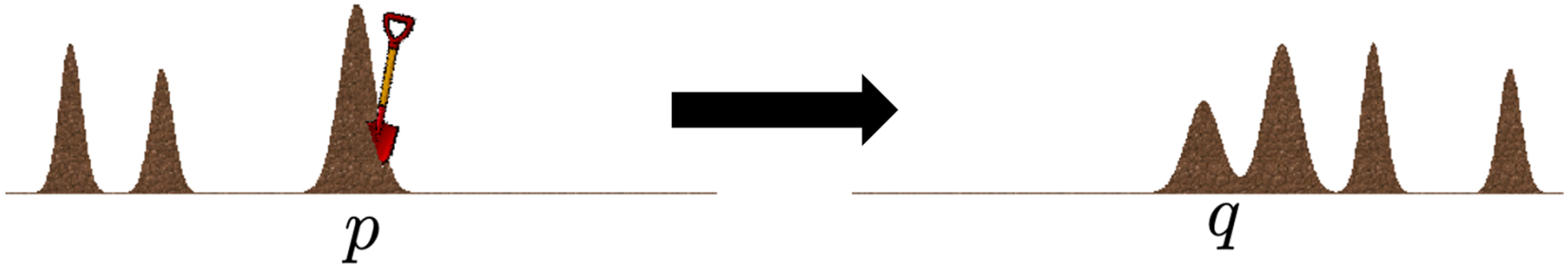


Can we have a measure like the green line?



Wasserstein GAN (WGAN)

- **Wasserstein Distance**



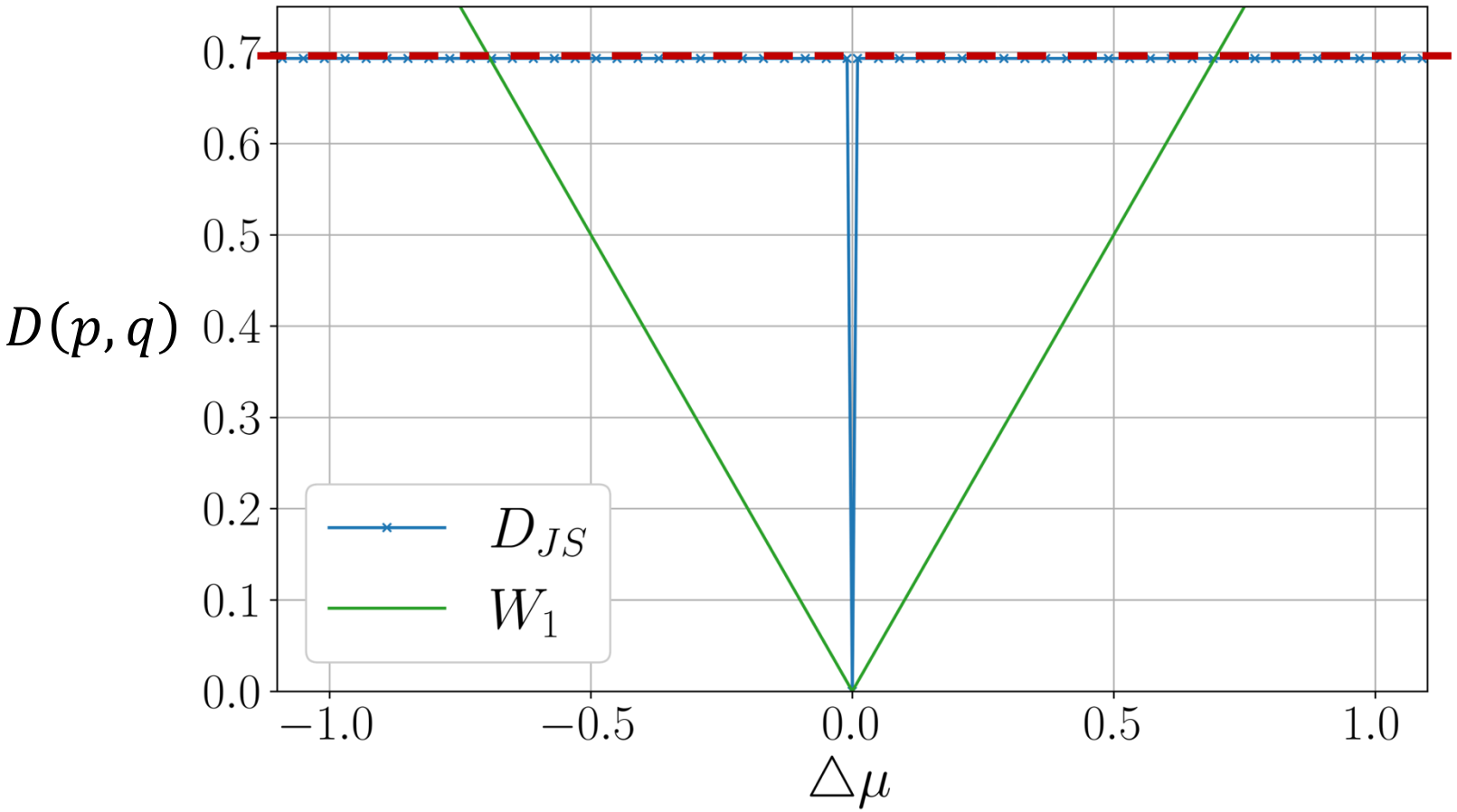
  - **"Earth Mover's Distance"**
    - ✓ If two distributions p and q are **"two piles of dirt"**, their distance is measured by **the cost of moving p to q**

$$\text{cost} = \text{mass} \times \text{distance}$$

    - ✓ **Do not saturate** when p and q does not overlap (distance is properly measured), as oppose to the JS Divergence

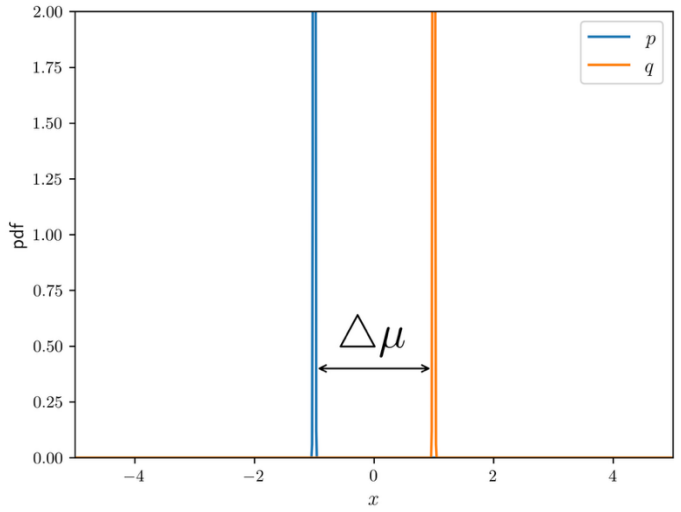
Wasserstein GAN (WGAN)

- **JS Divergence vs. Wasserstein Distance**
  - **Goal:** A **"smoother" distance** defined when p and q have **disjoint support**



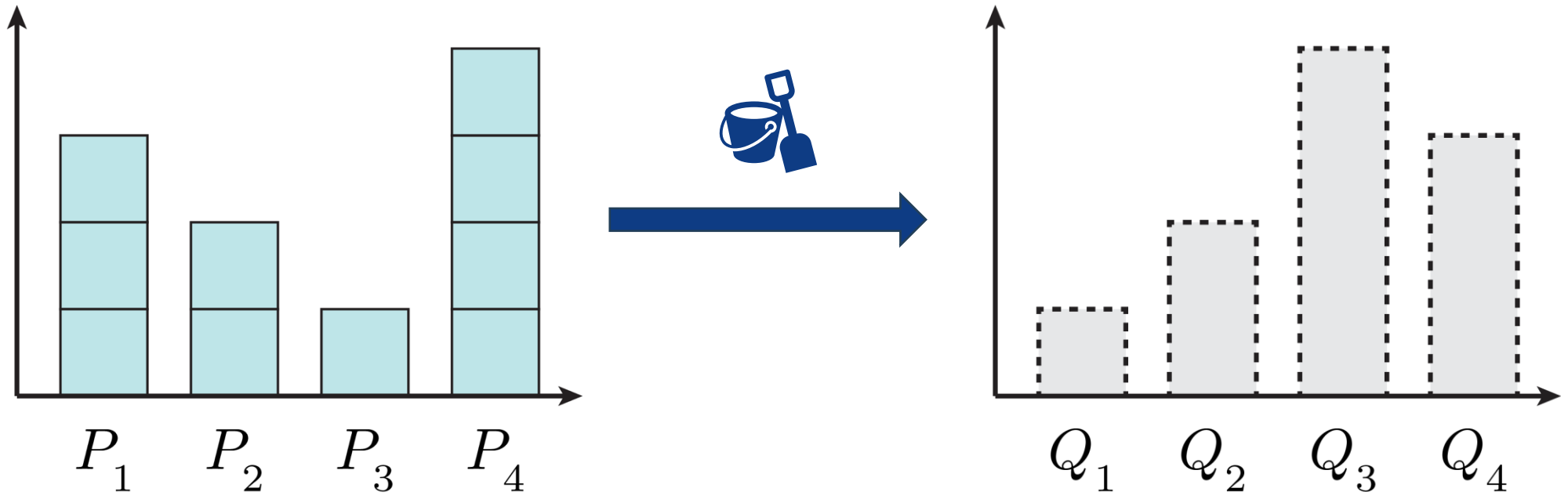
$$D_{JS}(p,q) = \log 2 \ (\mu_p \neq \mu_q)$$

$$\boldsymbol{W(p,q) = |\mu_p - \mu_q|}$$



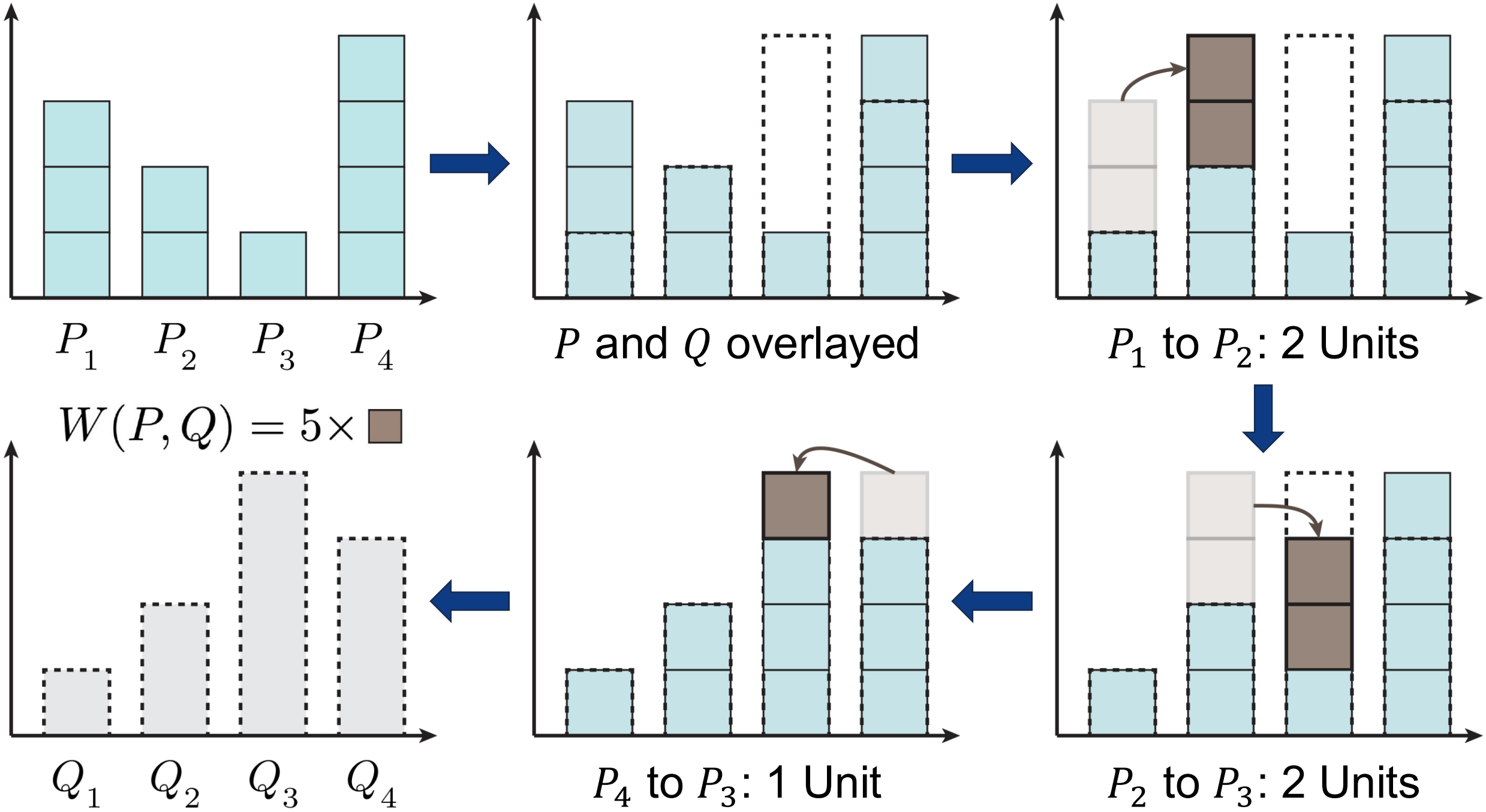
Wasserstein GAN (WGAN)

- **Wasserstein Distance: A Toy Example**
  - ➢ **"Earth Mover's Distance"**
    - ✓ **Question:** How to transport P to Q with minimum cost?



P and Q have the same amount of mass

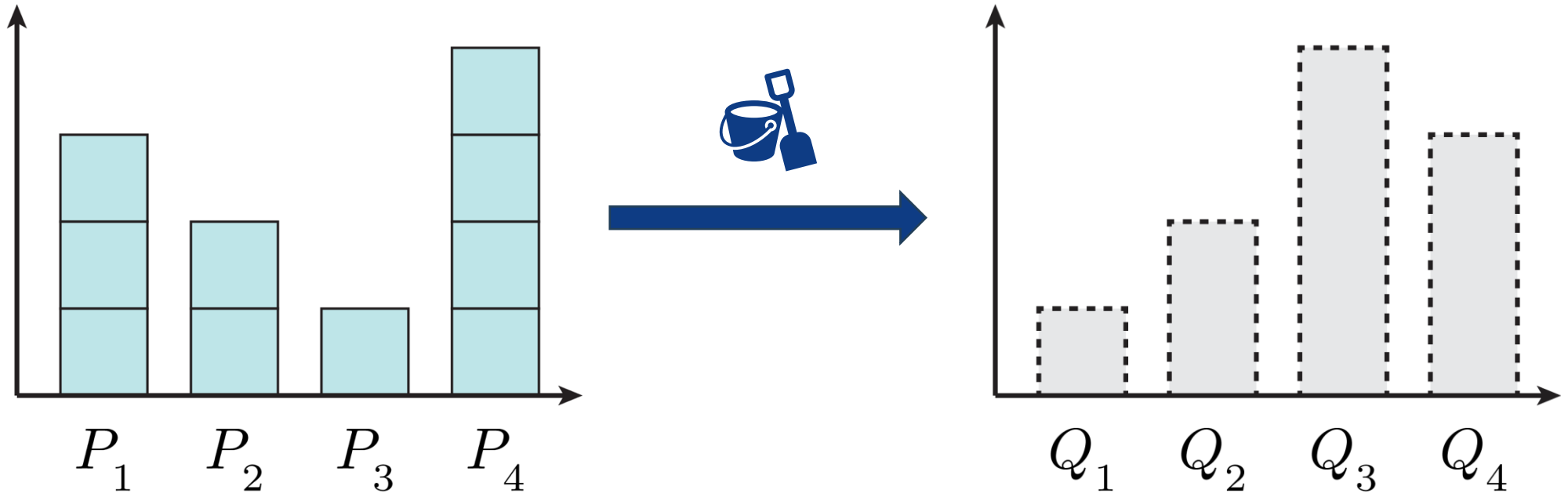
Example adapted from: Lilian Weng, "From GAN to WGAN", arXiv:1904.08994


P_1 P_2 P_3 P_4
P and Q overlayed
P_1 to P_2: 2 Units
P_2 to P_3: 2 Units
P_4 to P_3: 1 Unit
$W(P,Q) = 5\times$
Q_1 Q_2 Q_3 Q_4

Wasserstein GAN (WGAN)

- **Wasserstein Distance: Intuitive Understanding**
  - ➢ **"Earth Mover's Distance"**

$$W(P,Q) = \min_{\text{transport plans}} \text{total transport cost}$$



Example adapted from: Lilian Weng, "From GAN to WGAN", arXiv:1904.08994

Wasserstein GAN (WGAN)

- **Wasserstein Distance: Discrete Transport Plan**
  - **Definition: Transport plan matrix** $\Gamma = \{\gamma_{ij}\}$ $(\gamma_{ij} \geq 0)$, where

$$\gamma_{ij} = \text{mass moved from bin } i \text{ to bin } j$$

Such that

$$\sum_j \gamma_{ij} = P_i, \qquad \sum_i \gamma_{ij} = Q_j$$

Therefore, the Wasserstein distance $W(P, Q)$ can be written as

$$W(P, Q) = \min_{\Gamma} \sum_{i,j} \gamma_{ij} \cdot d(i, j)$$

where $d(i, j)$ is the transport distance $(\text{cost} = \text{mass} \times \text{distance})$

Wasserstein GAN (WGAN)

- **Wasserstein Distance: Continuous Definition**
  - **Informal Intuition:** $\Gamma = \{\gamma_{ij}\}$ can be considered as a **distribution coupling**

$$\gamma_{ij} = P(X = i, Y = j)$$

i.e., γ is a joint distribution with marginals P and Q

  - **Generalization:**

$$W(P,Q) = \min_{\Gamma \in \Pi(P,Q)} \sum_{i,j} \gamma_{ij}\ d(i,j)$$

Express as expectation ⇩

$$W(P,Q) = \min_{\Gamma \in \Pi(P,Q)} \mathbb{E}_{(X,Y)\sim\Gamma}\left[d(X,Y)\right]$$

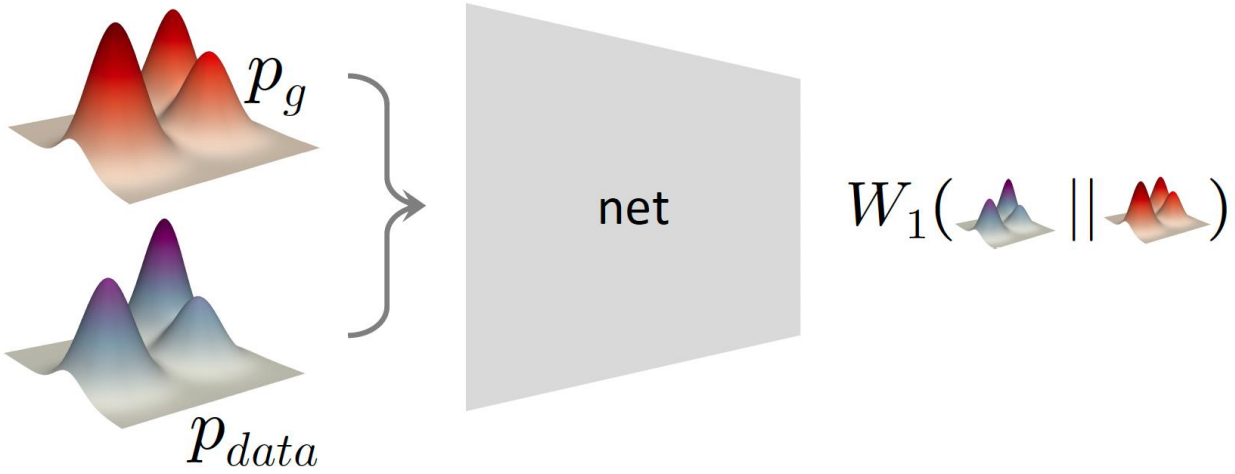
Extend to continuous field ⇩

$$W_1(p,q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(X,Y)\sim\gamma}\left[\|X - Y\|\right]$$

$\Pi(p,q)$:
Coupling of p and q

Wasserstein GAN (WGAN)

- **Wasserstein Distance in GAN**
  - Using Wasserstein Distance instead of JS Divergence in GAN

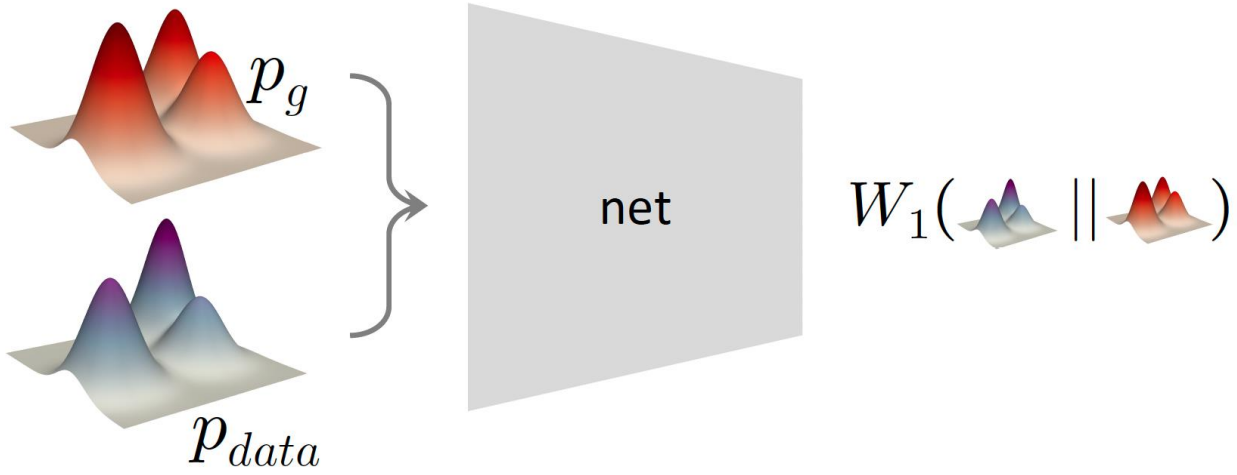


$$W_1(p, q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(X,Y)\sim\gamma} [\|X - Y\|]$$

$\Pi(p, q)$: Coupling of marginal distributions p and q

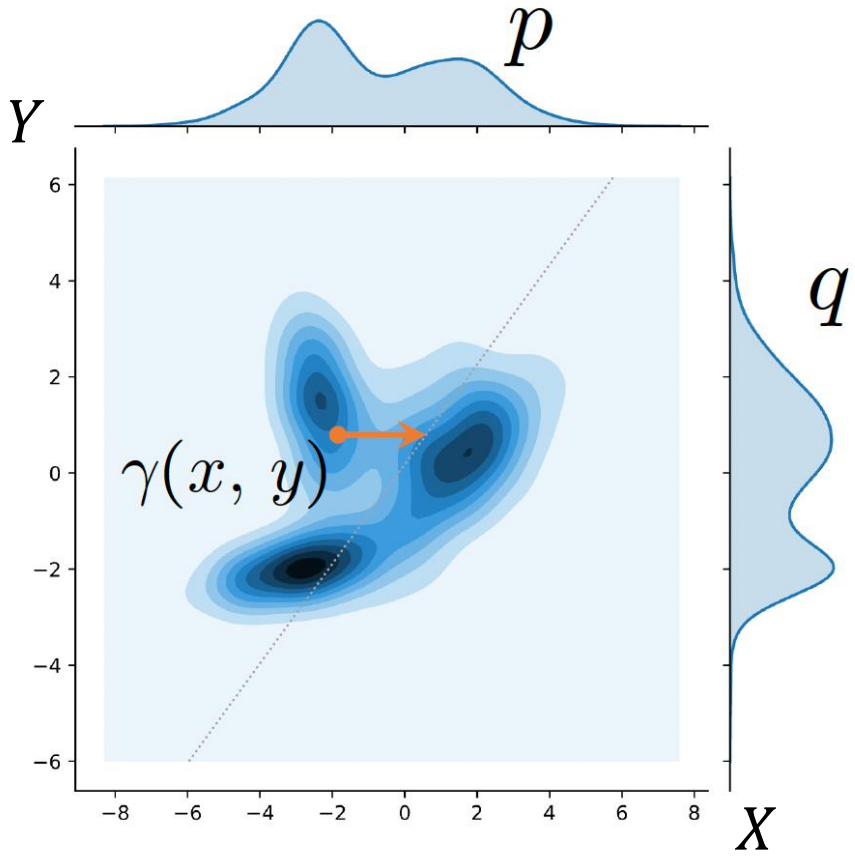
Wasserstein GAN (WGAN)

- **Wasserstein Distance in GAN**
  - Using Wasserstein Distance instead of JS Divergence in GAN



$$W_1(p, q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(X,Y)\sim\gamma} [\|X - Y\|]$$

$\Pi(p, q)$: Coupling of marginal distributions p and q



Wasserstein GAN (WGAN)

- **Wasserstein Distance in GAN**

$$W_1(p,q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(X,Y)\sim\gamma}\left[\|X - Y\|\right]$$

  - **Primal Problem:** Very difficult to solve for all transport plans $\gamma \in \Pi(p,q)$
  - By using **Kantorovich-Rubinstein duality**, we obtain the **dual problem**

$$W_1(p,q) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{x\sim p}[f(x)] - \mathbb{E}_{x\sim q}[f(x)]\right)$$

where $\|f\|_L \leq 1$ means the **critic function** $f(\cdot)$ is **1-Lipschitz**:

$$\forall x, y: \quad |f(x) - f(y)| \leq \|x - y\|_1$$

Intuitively, $f(\cdot)$ cannot change too fast (constrained gradient)

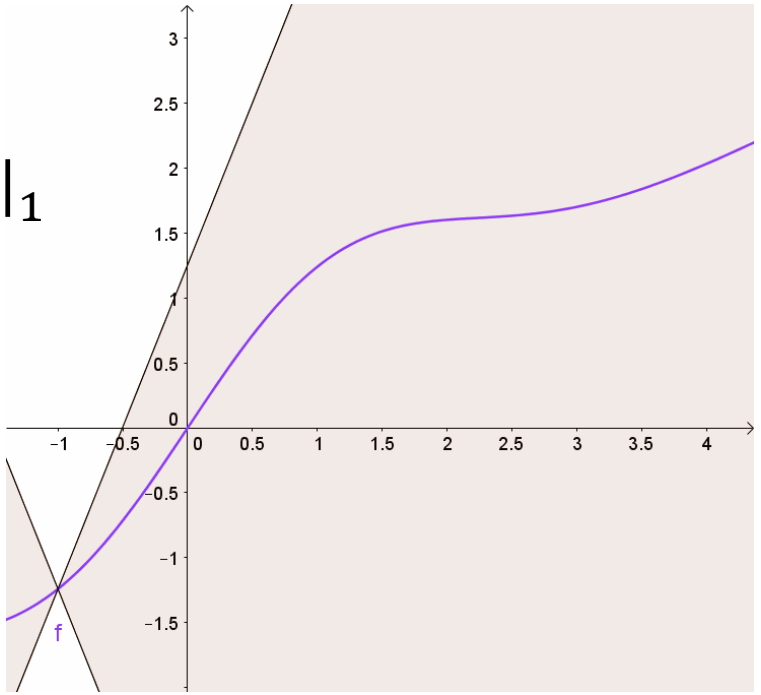
Wasserstein GAN (WGAN)

- **Wasserstein Distance in GAN**
  - By using **Kantorovich-Rubinstein duality**, we obtain the **dual problem**

$$W_1(p, q) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{x \sim q}[f(x)] \right)$$

where $\|f\|_L \leq 1$ means $f(\cdot)$ is **1-Lipschitz**:

$$\forall x, y: \quad |f(x) - f(y)| \leq \|x - y\|_1$$



  - In practice, "Lipschitzness" is implemented via
    - ✓ **Weight Clipping**
    - ✓ **Gradient Penalty** (Regularization Loss)

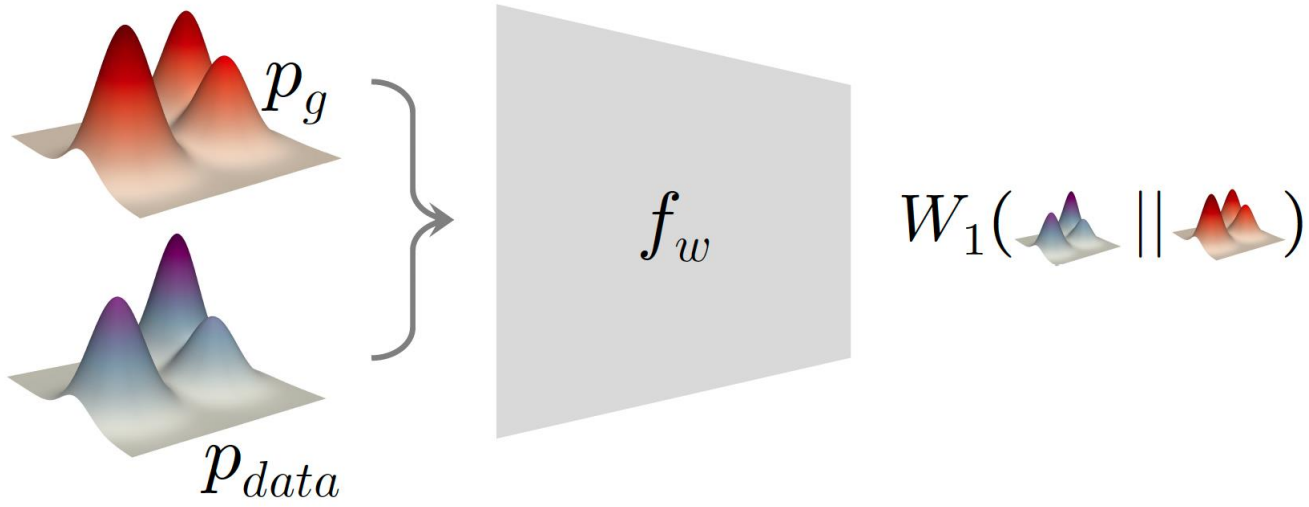
Gulrajani et al., "Improved Training of Wasserstein GANs," NeurIPS 2017.

Wasserstein GAN (WGAN)

- **Wasserstein GAN: Objective Function**

$$W_1(p, q) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{x \sim q}[f(x)] \right)$$

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{\text{data}}}[f_w(x)] - \mathbb{E}_{x \sim p_g}[f_w(x)]$$



- f_w: Parameterized via neural network

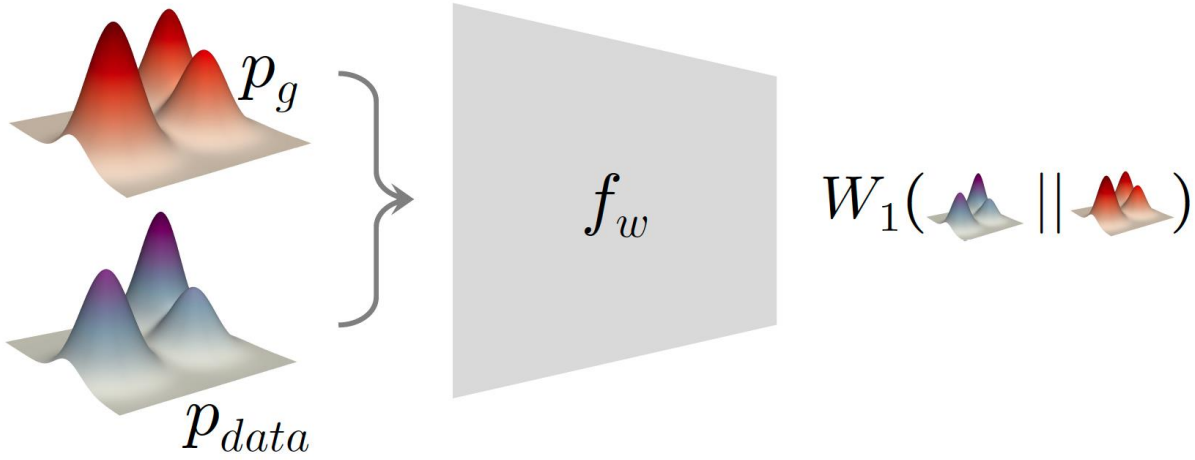
  ➔ **"Critic" Model**

- $w \in \mathcal{W}$: Weights of f_w satisfies 1-Lipschitz

Wasserstein GAN (WGAN)

- **Wasserstein GAN: Optimization**

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{data}} [f_w(x)] - \mathbb{E}_{x \sim p_g} [f_w(x)]$$



➢ **Critic Model f_w:**

✓ Assign higher score to $x \sim p_{data}$ (“real” data)

✓ Assign lower score to $x \sim p_g$ (“fake” data)

} **“Realness Scorer”**

Wasserstein GAN (WGAN)

- **W-GAN vs. Original GAN**
  - **Original GAN**

$$\max_{D} \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] - \mathbb{E}_{x\sim p_g}\left[\log\left(1 - D(x)\right)\right]$$

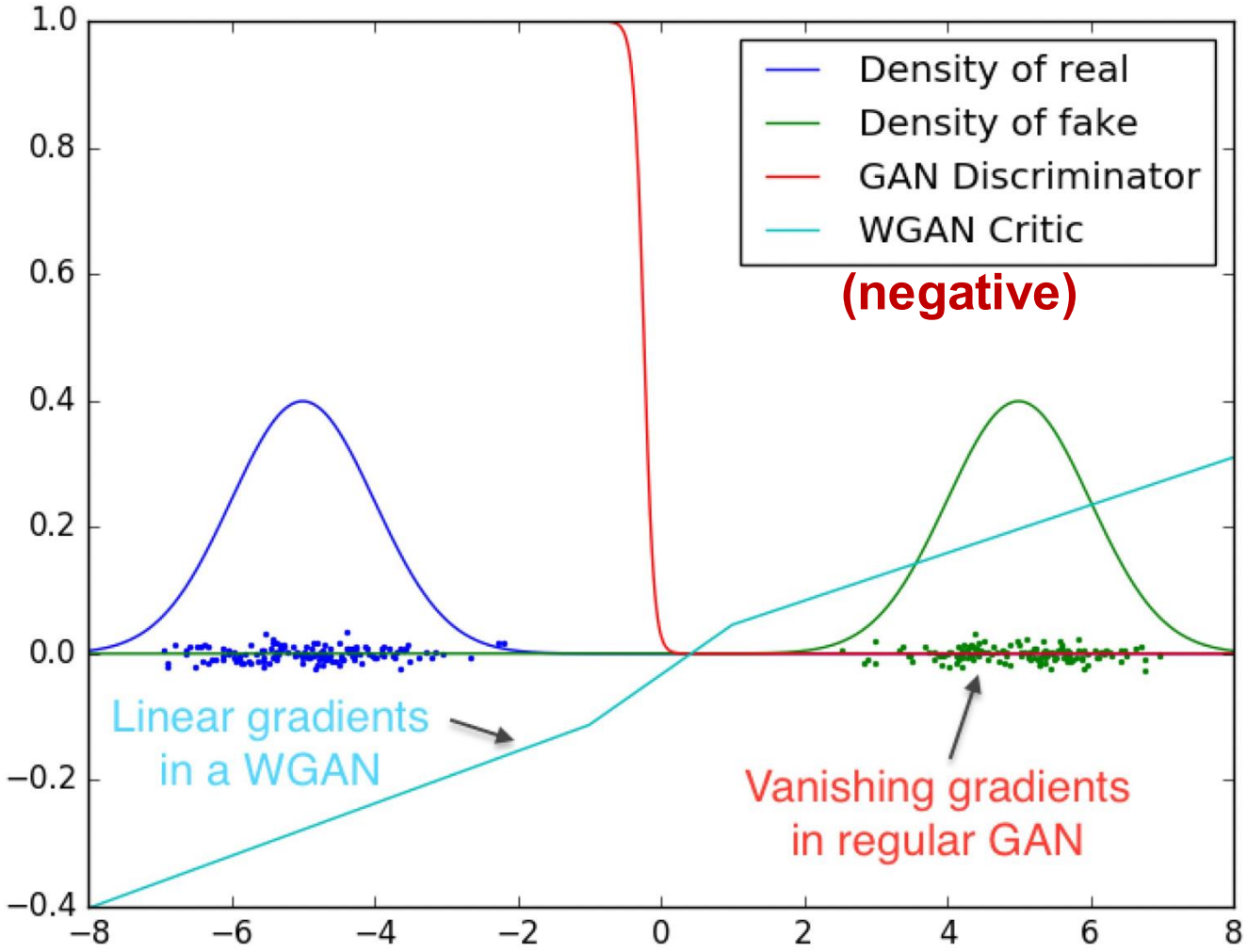
    - ✓ **"Forgery Expert": real or fake**
  - **W-GAN**

$$\max_{w\in\mathcal{W}} \mathbb{E}_{x\sim p_{\text{data}}}[f_w(x)] - \mathbb{E}_{x\sim p_g}[f_w(x)]$$

    - ✓ No logarithms
    - ✓ 1-Lipschitz Constraints
    - ✓ **"Art Critic": score / merit / quality ➔ direction to improve (gradient)**

Wasserstein GAN (WGAN)

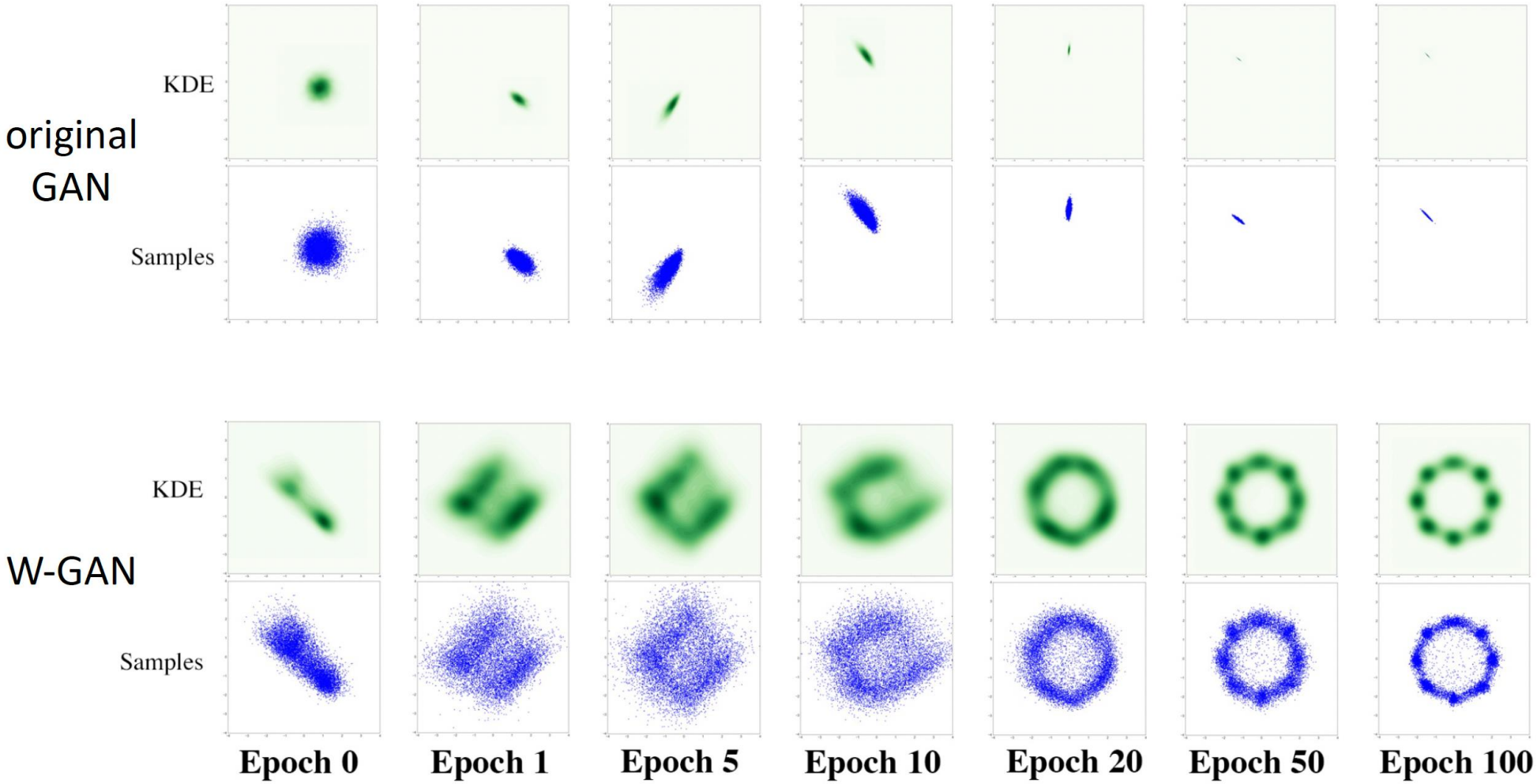
- **W-GAN vs. Original GAN**



Restricted by 1-Lipschitz

Wasserstein GAN (WGAN)

- **W-GAN vs. Original GAN**



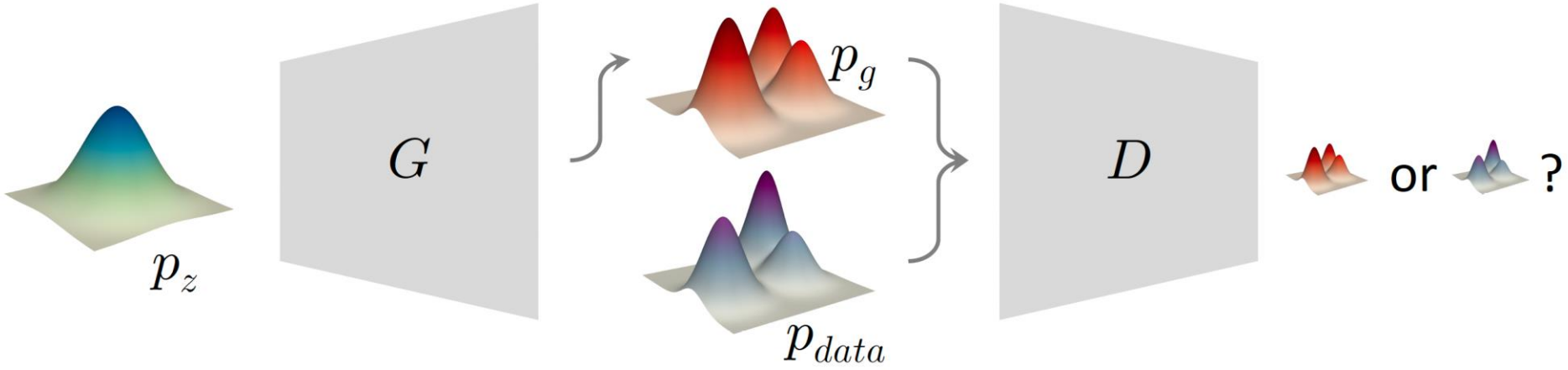


Lecture Outline

- **Wasserstein GAN (WGAN)**
- **Adversary as a Loss Function**
- **“Scaling” of GAN Models**

Adversary as a Loss Function

- **Key Motivation**
  - ➢ Essentially, GAN defines an **adversarial loss function**
  - ➔ A **versatile framework** can cooperate with / applied to many other…
    - ✓ **Loss functions** (e.g., reconstruction loss)
    - ✓ **Generative models** (e.g., VAE, AR)
    - ✓ **Practical tasks** (e.g., domain adaptation)

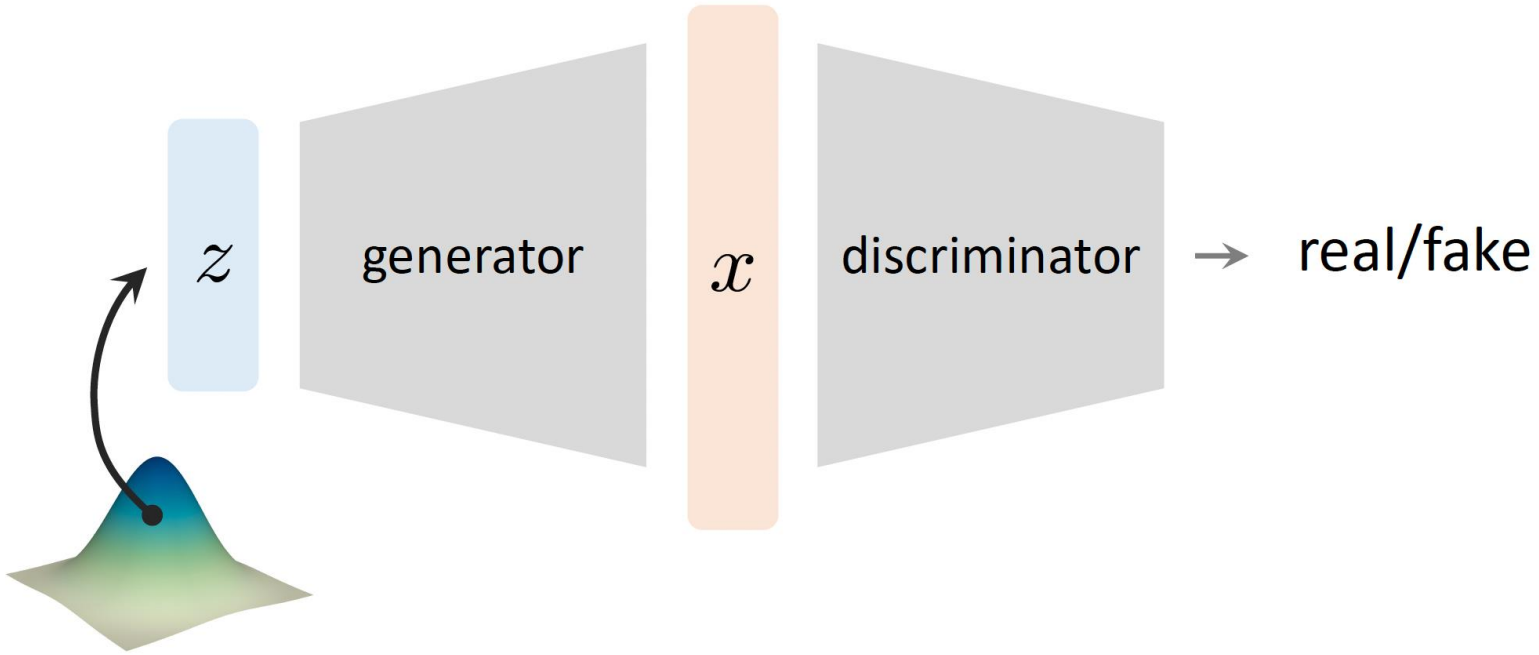


Adversary as a Loss Function

- **Alternative Input**
  - ➢ So far, GAN receives random noise $z \sim p(z)$ as input

$$x = G(z), \qquad x \rightarrow D(x)$$

which is known as **"Unconditional Generation"**

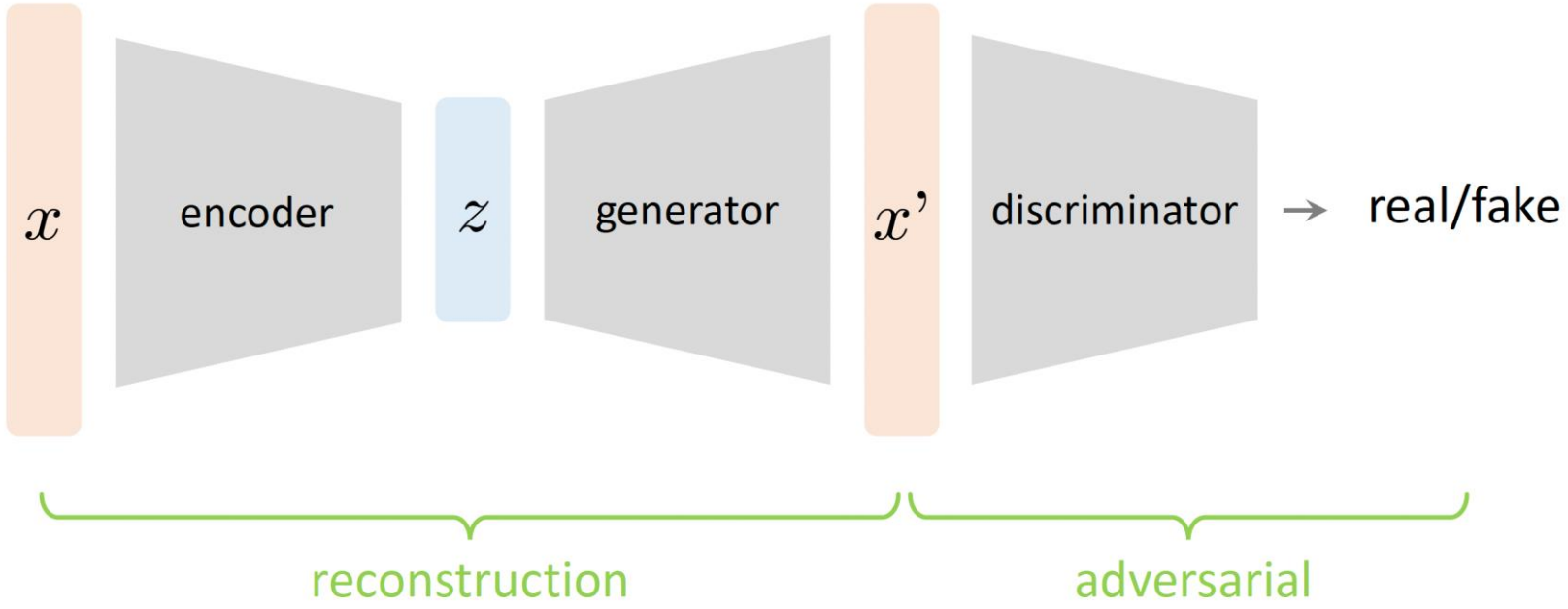


Adversary as a Loss Function

- **Alternative Input**
  - However, input can be from any other sources x

$$z = E(x), x' = G(z), \qquad x \,/\, x' \rightarrow D(\cdot)$$

  - **"Conditional Generation"**: Generation conditioned on given input x



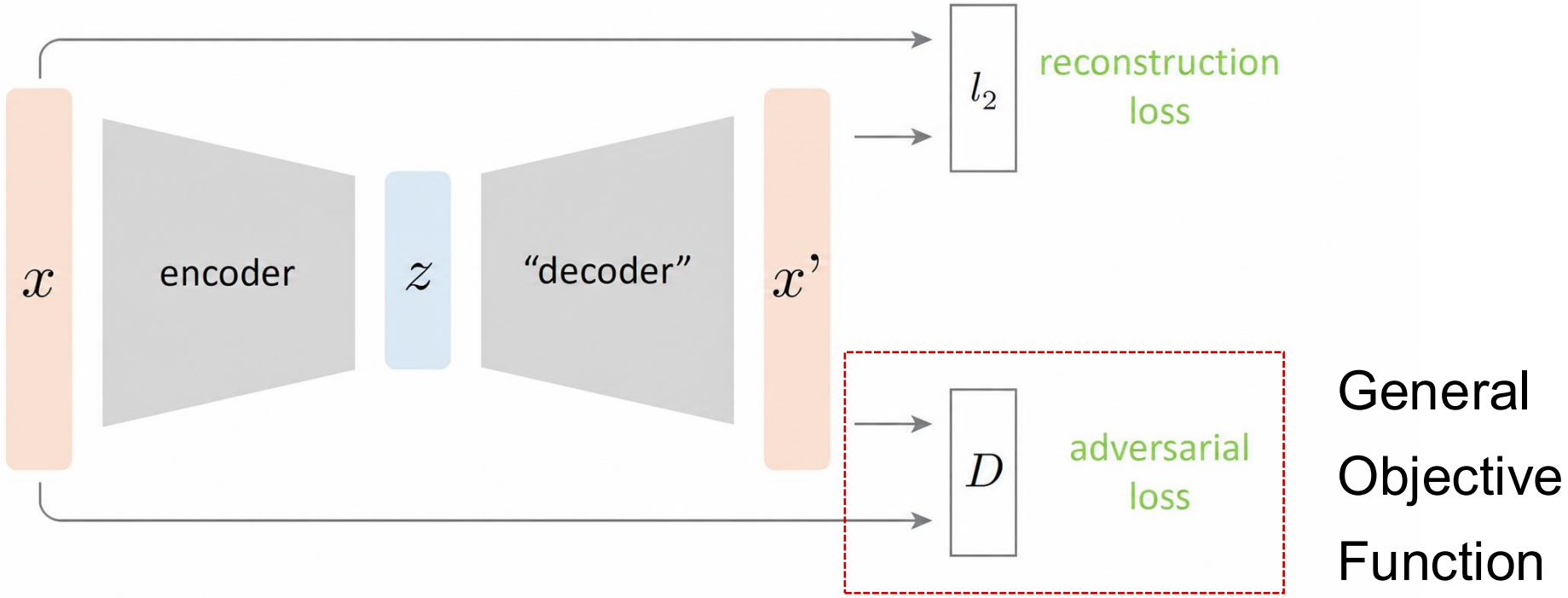
Improving the visual quality of x by reconstruction

Adversary as a Loss Function

- **Alternative Input**
  - However, input can be from any other sources x

$$z = E(x), x' = G(z), \qquad x \,/\, x' \rightarrow D(\cdot)$$

  - **"Conditional Generation"**: Generation conditioned on given input x



General Objective Function

Adversary as a Loss Function

- **Example: Super-Resolution GAN (SRGAN)**

Worse PSNR

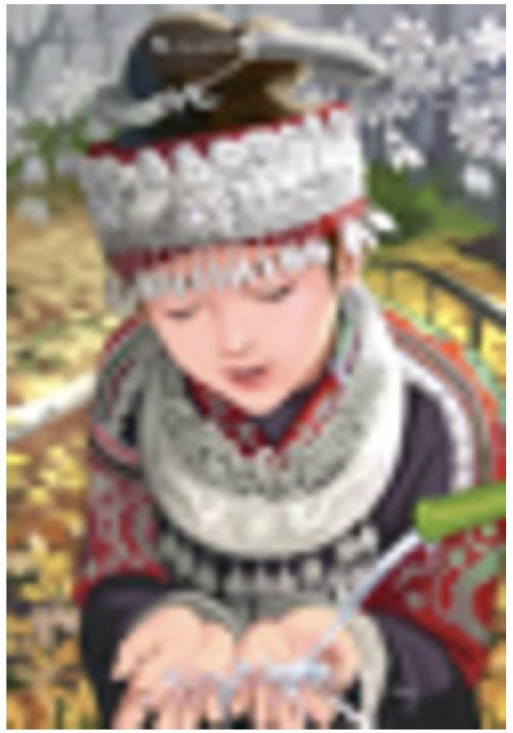
Better PSNR

Better Quality

original



bicubic
(21.59dB/0.6423)



SRResNet
(23.44dB/0.7777)



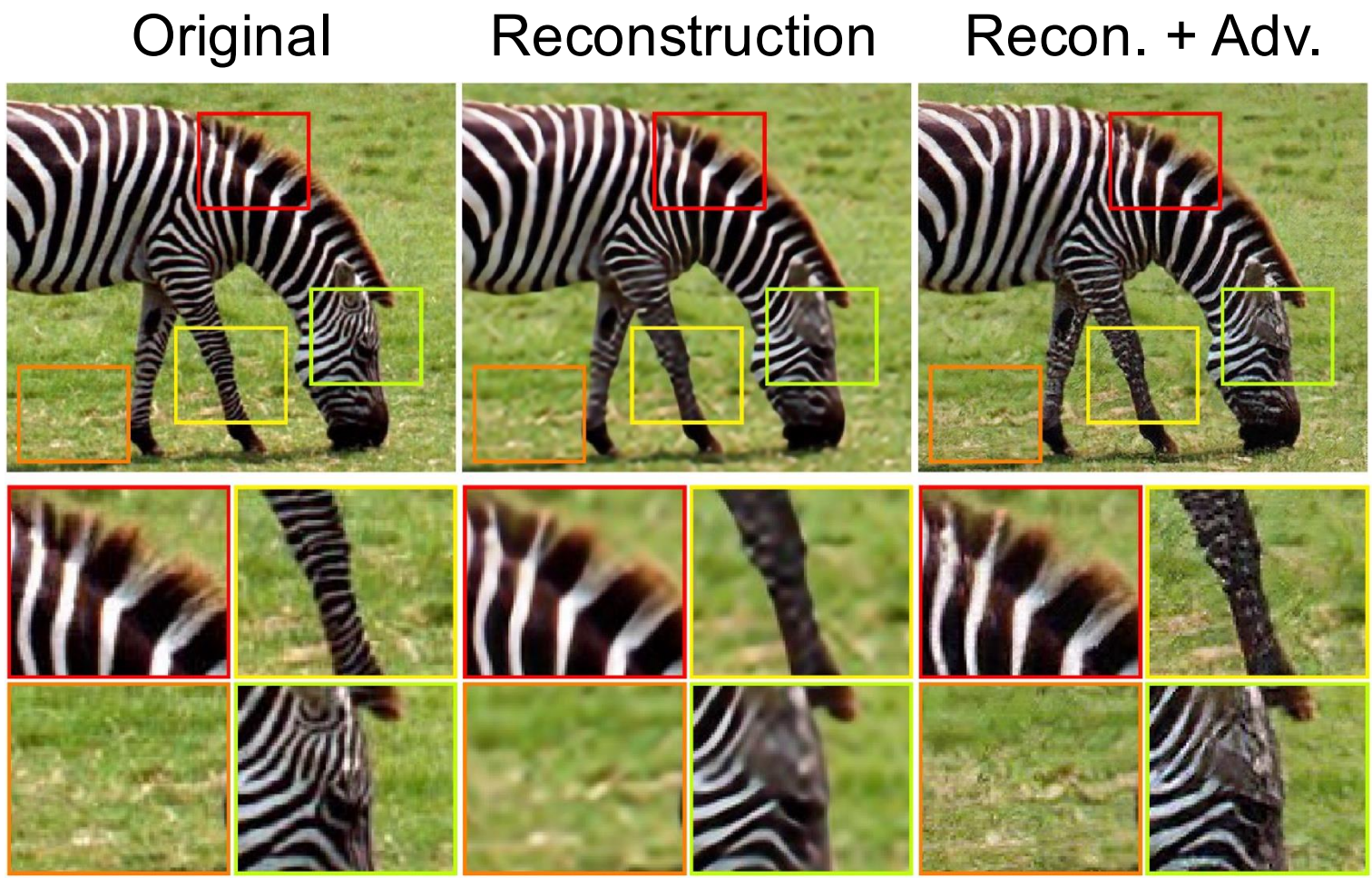
SRGAN
(20.34dB/0.6562)



Ledig, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network”, CVPR 2016

Adversary as a Loss Function

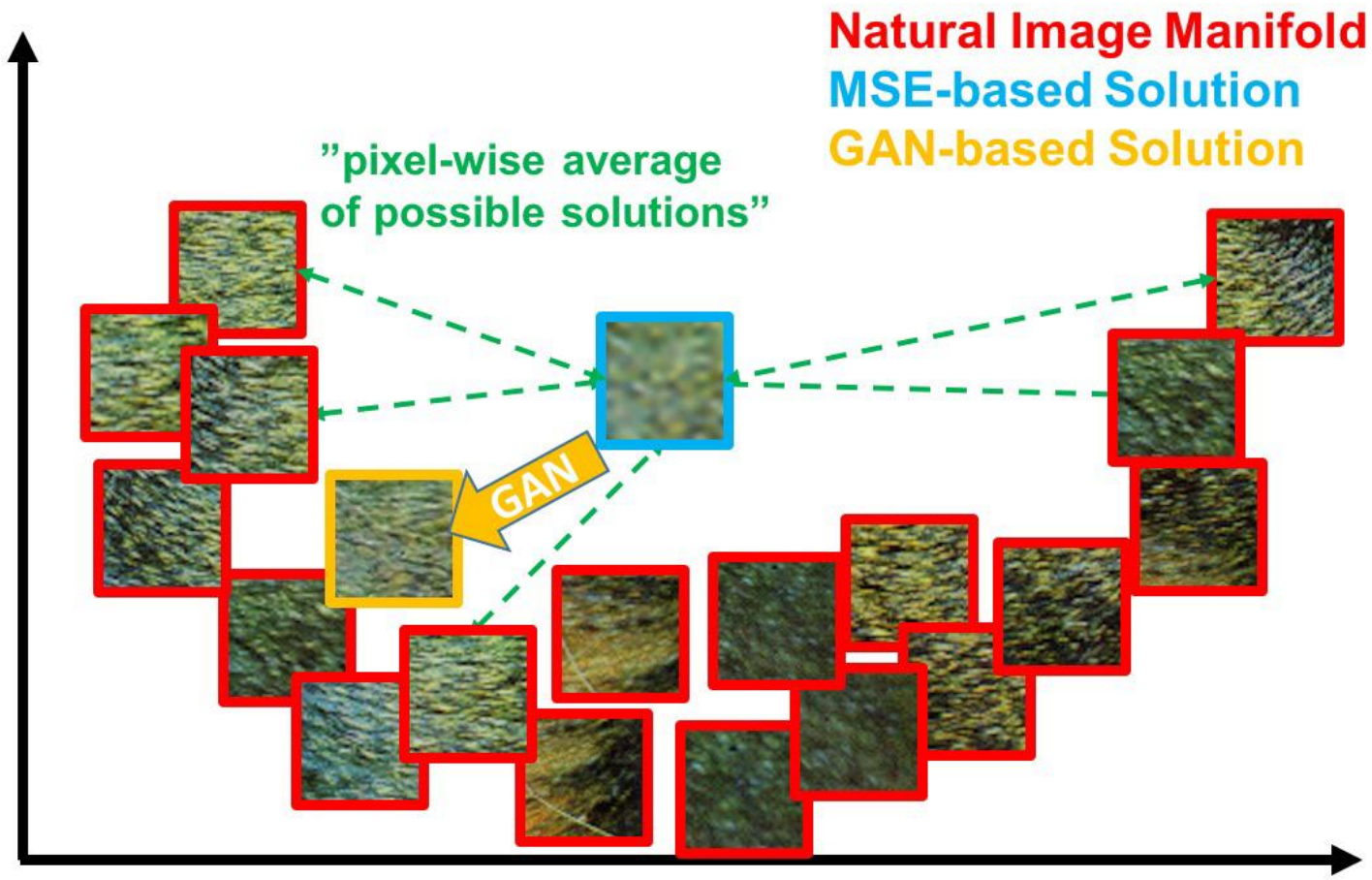
- **Example: Super-Resolution GAN (SRGAN)**



Ledig, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network”, CVPR 2016

Adversary as a Loss Function

- **Example: Super-Resolution GAN (SRGAN)**

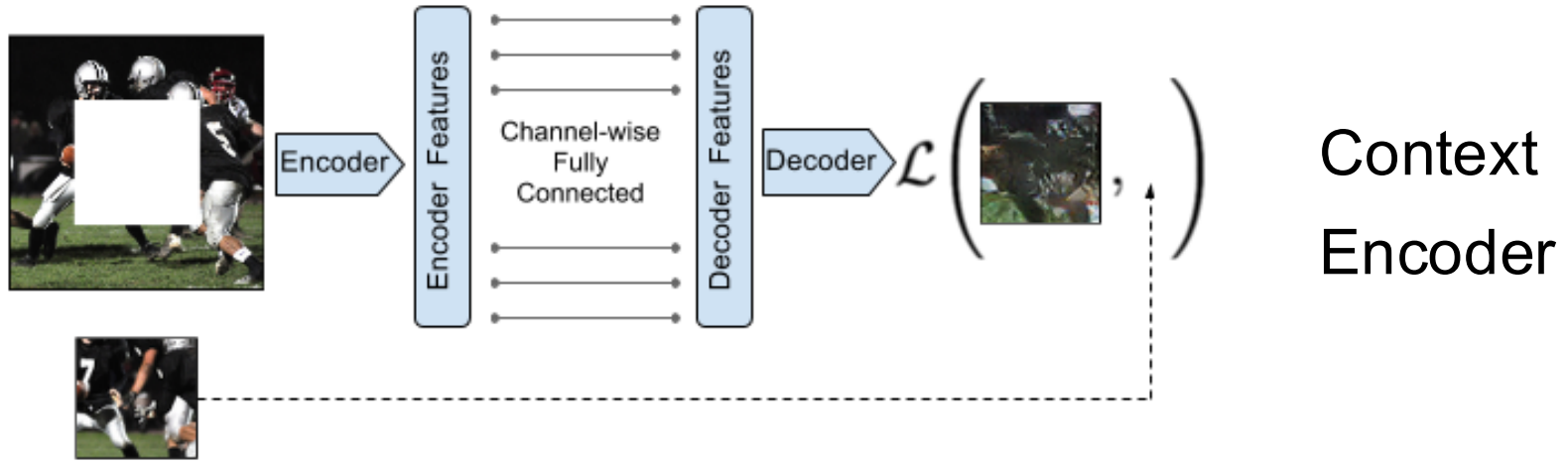


Ledig, et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network", CVPR 2016

Adversary as a Loss Function

- **Example: Image Inpainting**

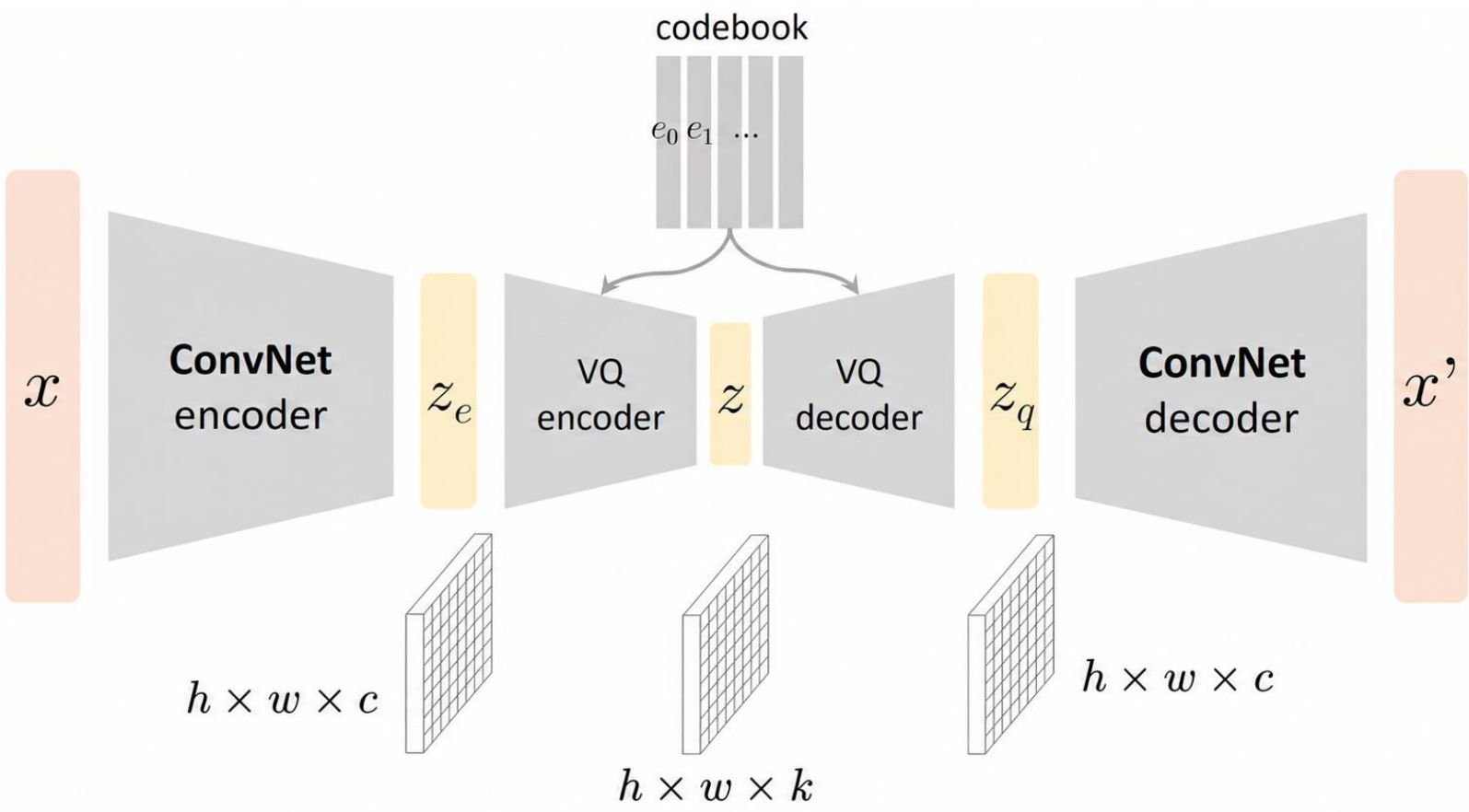




Pathak, et al., "Context Encoders: Feature Learning by Inpainting", CVPR 2016

Adversary as a Loss Function

- **Example: Visual Tokenizer (VQ-GAN)**



VQ-VAE:

L2 loss only



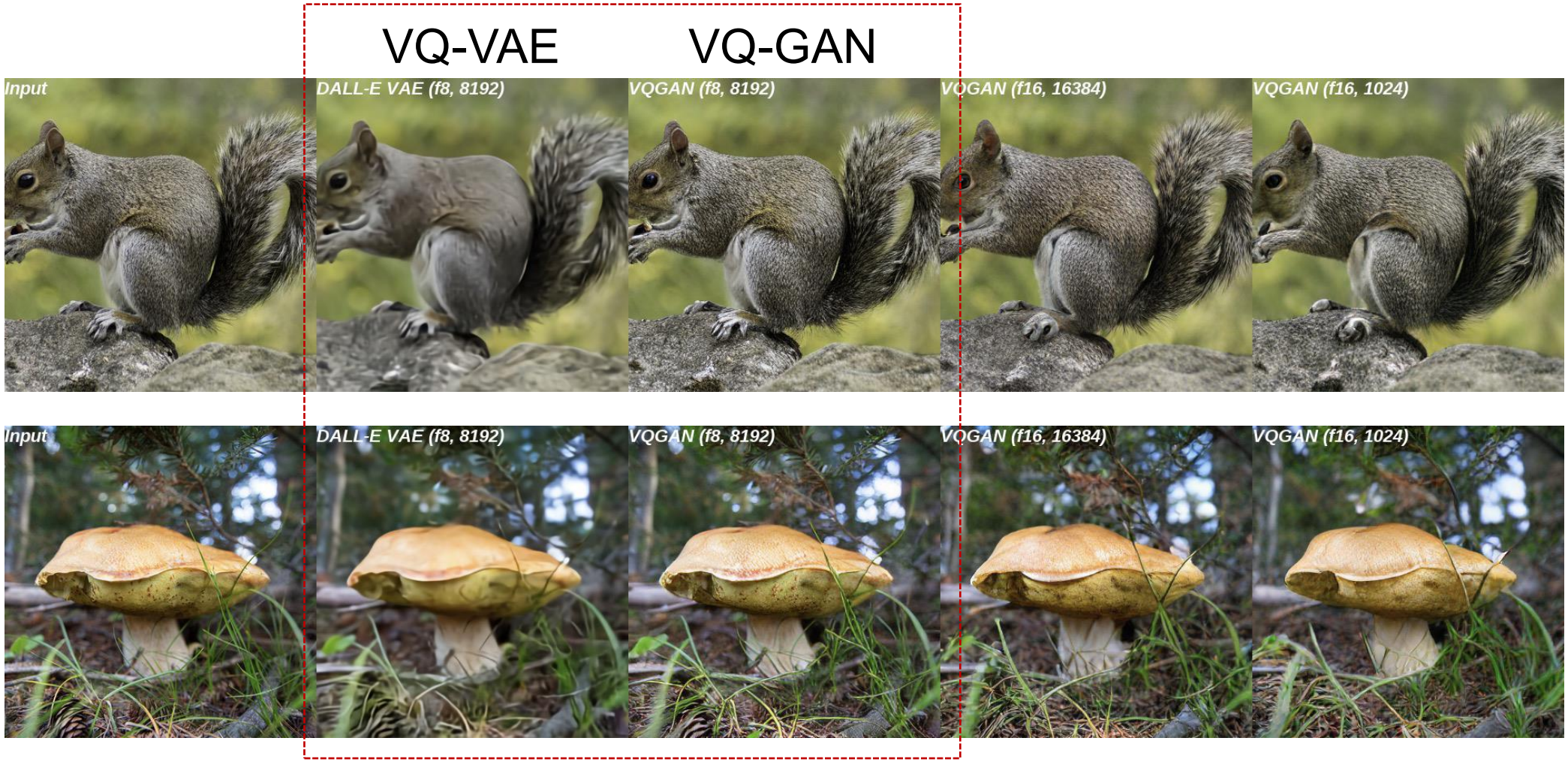
VQ-GAN:

L2 + Adv. loss

Esser, et al., "Taming Transformers for High-Resolution Image Synthesis", CVPR 2021

Adversary as a Loss Function

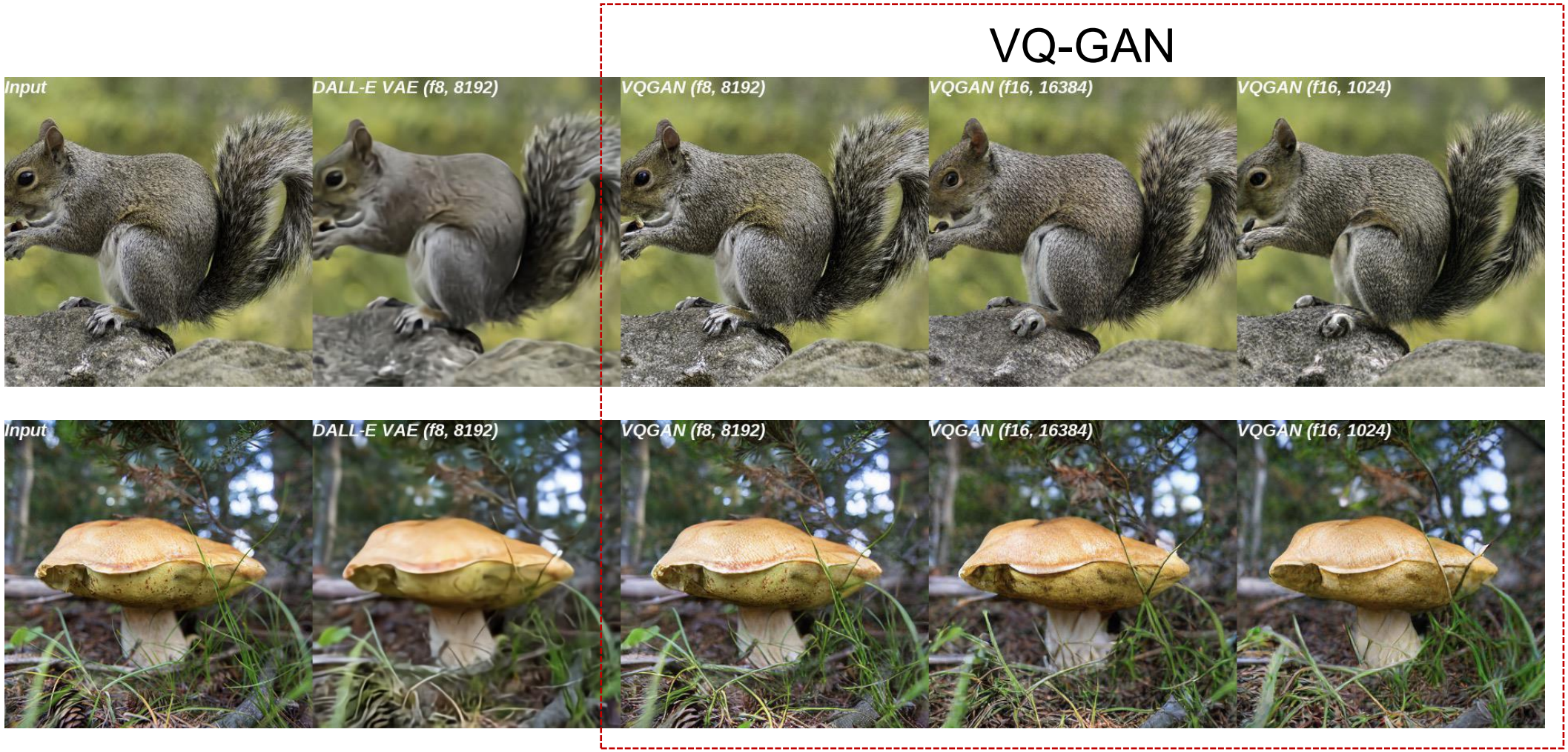
- **Example: Visual Tokenizer (VQ-GAN)**



Esser, et al., "Taming Transformers for High-Resolution Image Synthesis", CVPR 2021

Adversary as a Loss Function

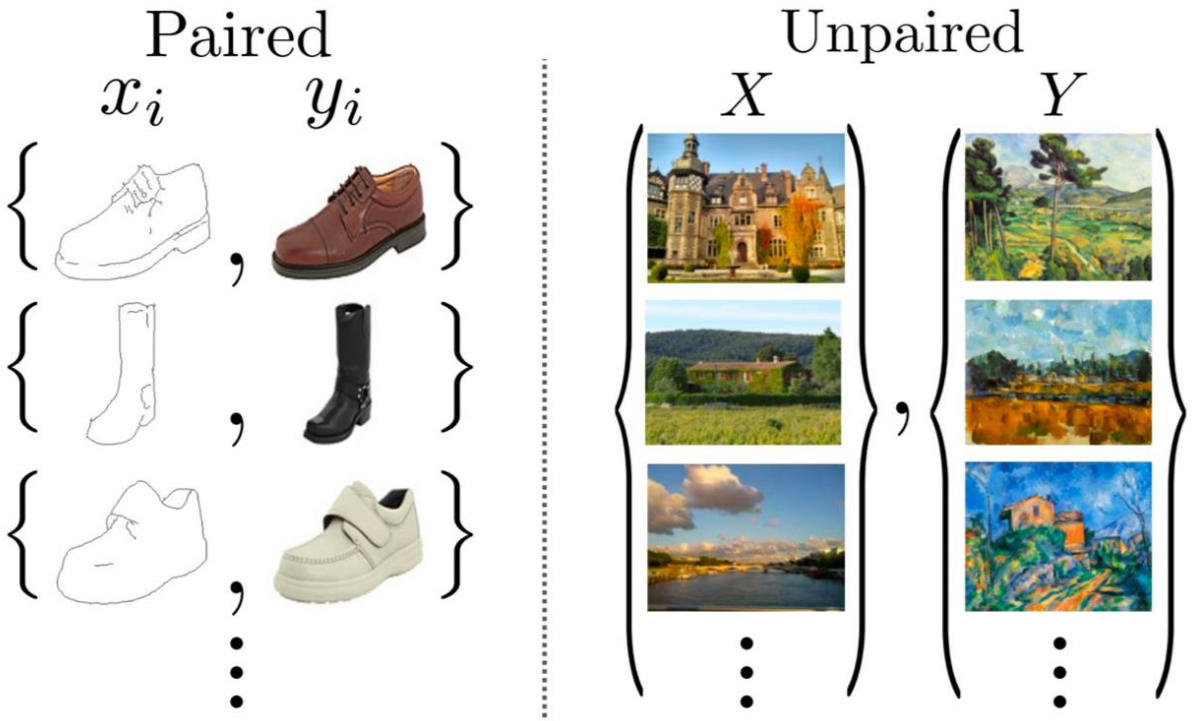
- **Example: Visual Tokenizer (VQ-GAN)**



Esser, et al., "Taming Transformers for High-Resolution Image Synthesis", CVPR 2021

Adversary as a Loss Function

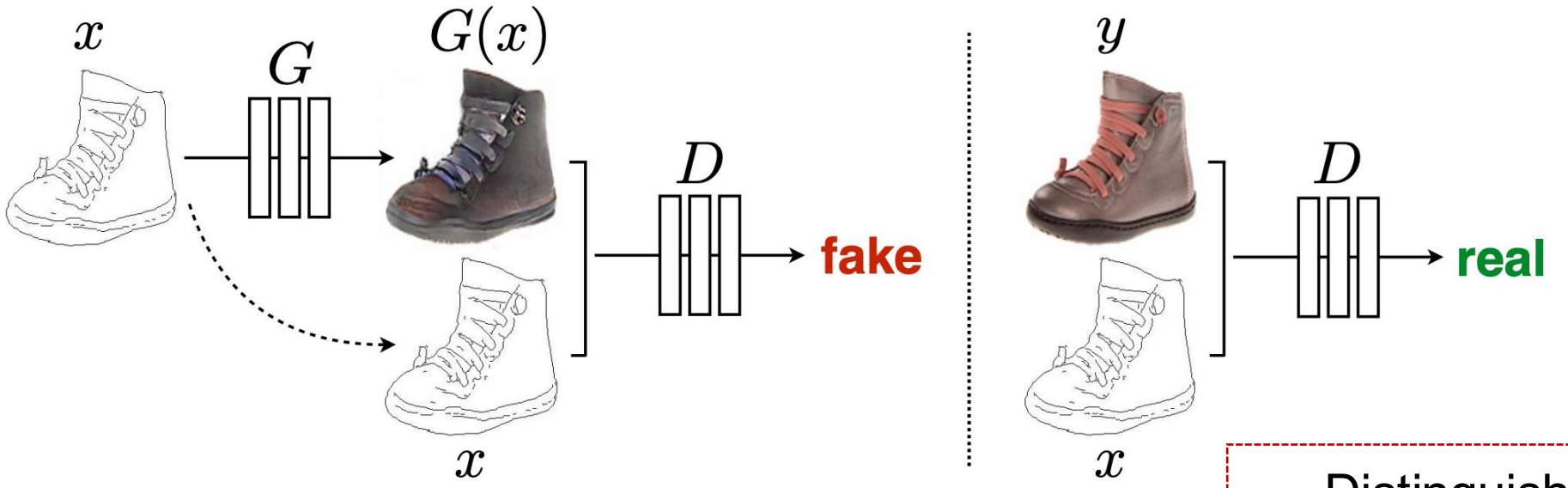
- **Example: Image Translation**
  - ➢ **Definition:** Map images from one domain to another ($\mathcal{X} \rightarrow \mathcal{Y}$)
    - ✓ Mixing content/semantic from $\mathcal{X}$ and style from $\mathcal{Y}$ ("**Style Transfer**")



Zhu, et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks", ICCV 2017

Adversary as a Loss Function

- **Example: Paired Image Translation (Pix2Pix)**
  - **Definition:** Map images from one domain to another ($\mathcal{X} \rightarrow \mathcal{Y}$)
    - ✓ Mixing content/semantic from $\mathcal{X}$ and style from $\mathcal{Y}$ ("**Style Transfer**")



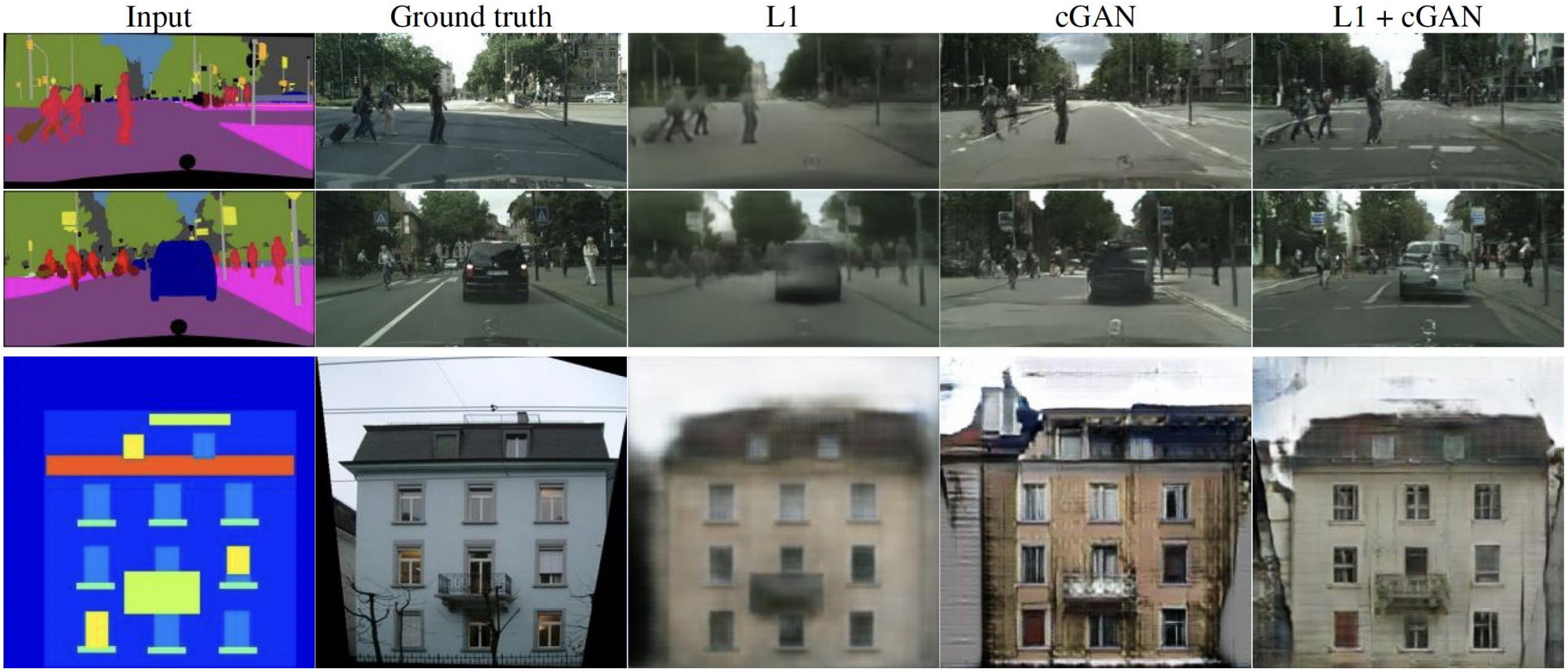
Framework of Pix2Pix

Distinguish real / fake (edge, photo) **image tuples**

Isola, et al., "Image-to-Image Translation with Conditional Adversarial Networks", CVPR 2017

Adversary as a Loss Function

- **Example: Paired Image Translation (Pix2Pix)**

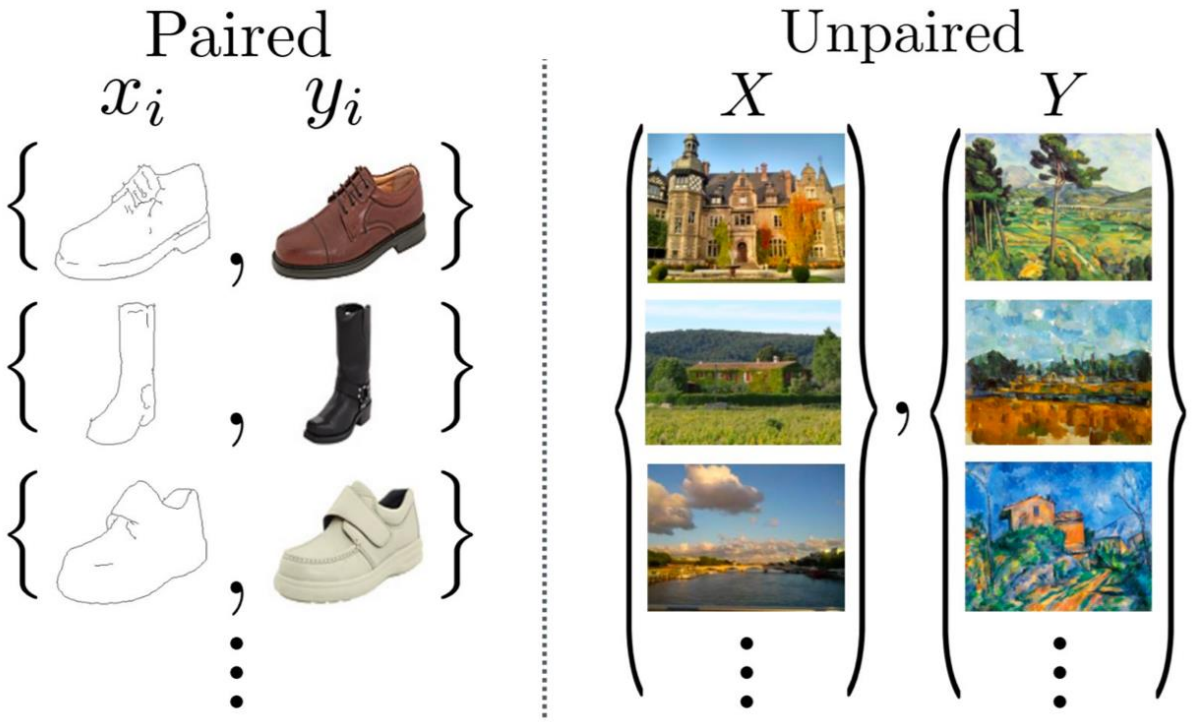


Isola, et al., “Image-to-Image Translation with Conditional Adversarial Networks”, CVPR 2017

Adversary as a Loss Function

- **Example: Image Translation**
  - **Definition:** Map images from one domain to another ($\mathcal{X} \rightarrow \mathcal{Y}$)
    - ✓ Mixing content/semantic from $\mathcal{X}$ and style from $\mathcal{Y}$ ("**Style Transfer**")

Too Expensive (even impossible) to Obtain

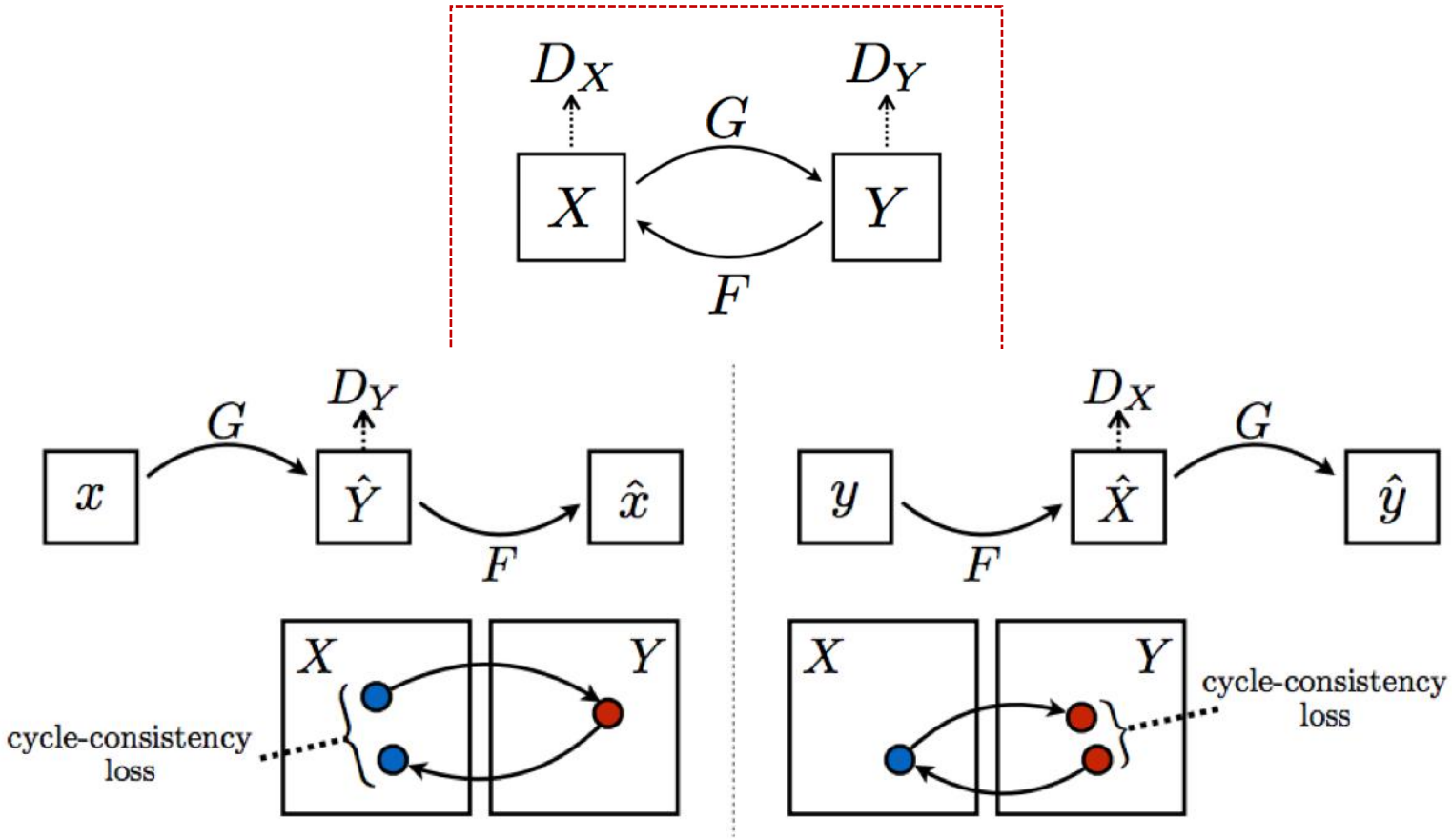


No Ground-Truth Provided

Zhu, et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks", ICCV 2017

Adversary as a Loss Function

- **Example: Unpaired Image Translation (CycleGAN)**
  - **Motivation:** Learn a reverse mapping to enforce **cyclic consistency**

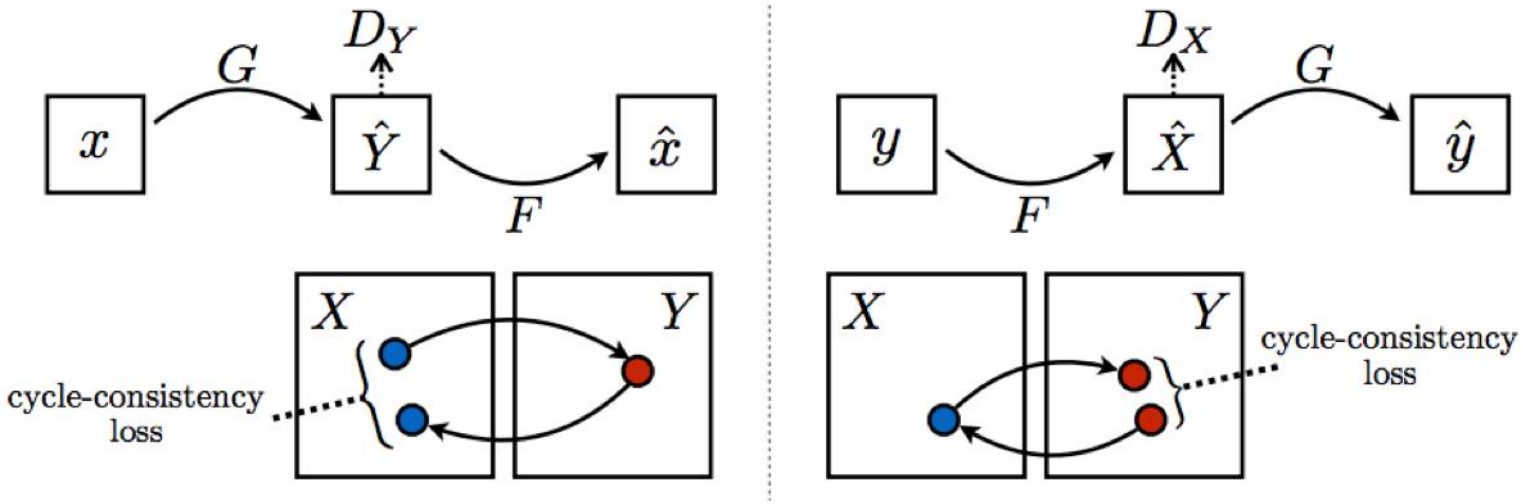


Zhu, et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks", ICCV 2017

Adversary as a Loss Function

- **Example: Unpaired Image Translation (CycleGAN)**
  - **Motivation:** Learn a reverse mapping to enforce **cyclic consistency**

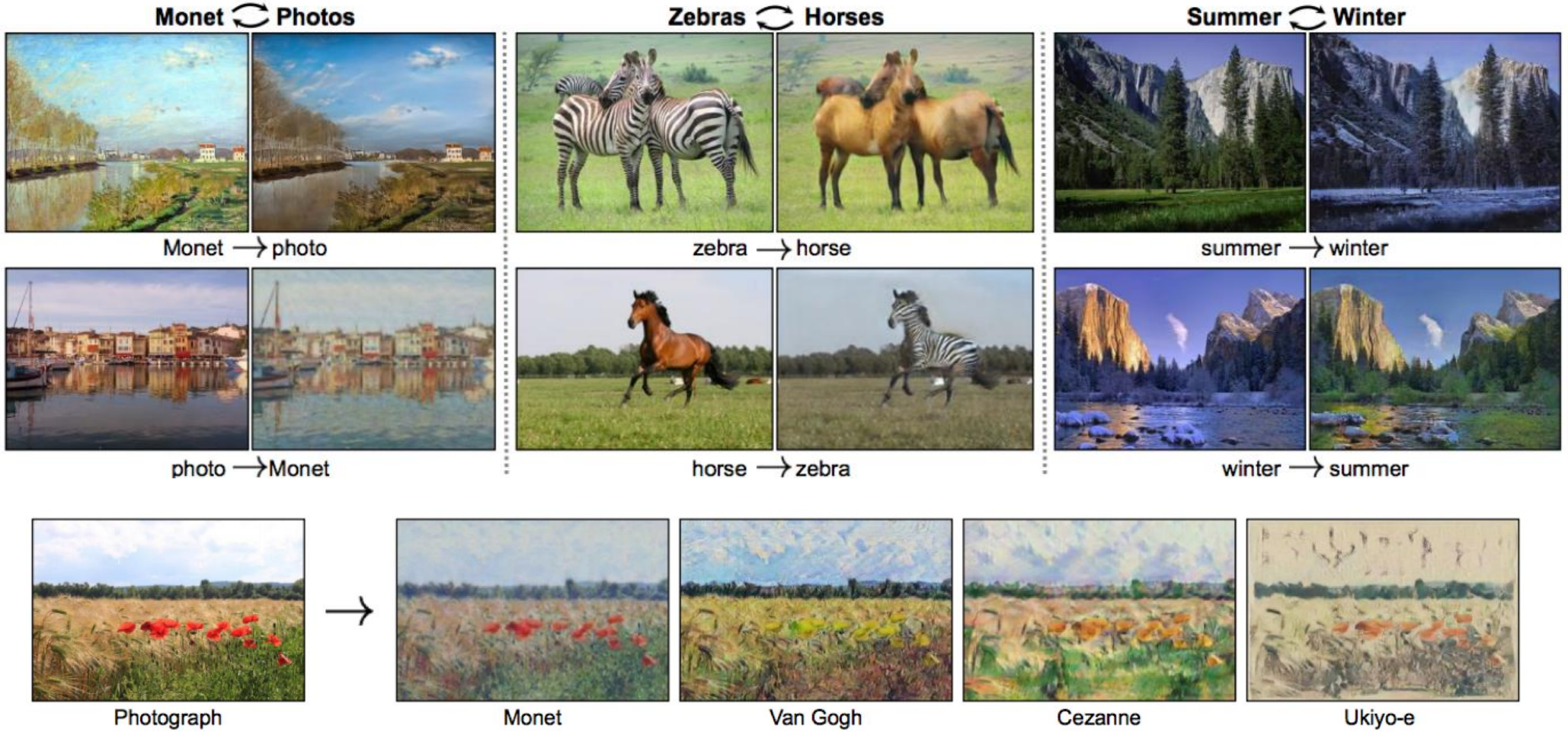
$$\min_{F,G,D_{\mathcal{X}},D_{\mathcal{Y}}} \mathcal{L}_{\text{GAN}}(G, D_{\mathcal{Y}}, X, Y) + \mathcal{L}_{\text{GAN}}(F, D_{\mathcal{X}}, X, Y)$$
$$+\lambda \underbrace{\left(E_X[\|F(G(X)) - X\|_1] + E_Y[\|G(F(Y)) - Y\|_1]\right)}_{\text{cycle consistency}}$$



Zhu, et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks", ICCV 2017

Adversary as a Loss Function

- **Example: Unpaired Image Translation (CycleGAN)**



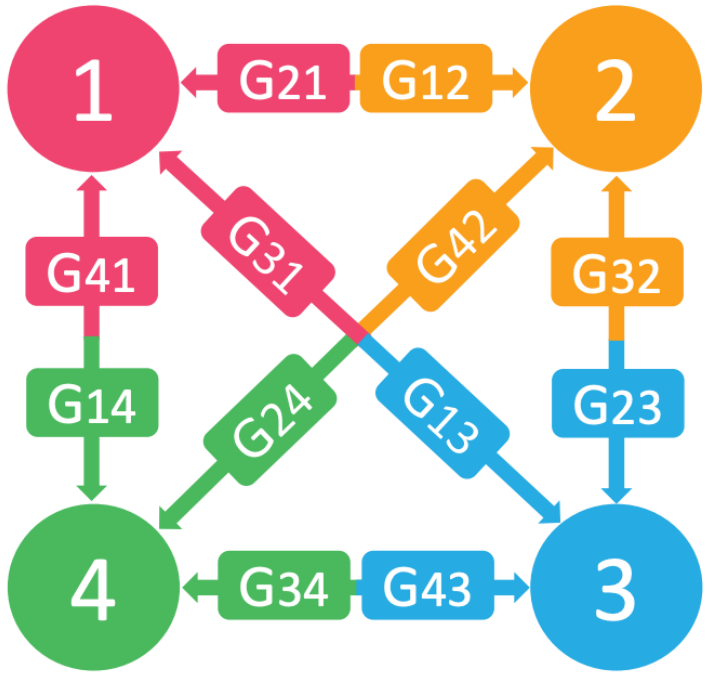
Source: Zhu et al., 2016

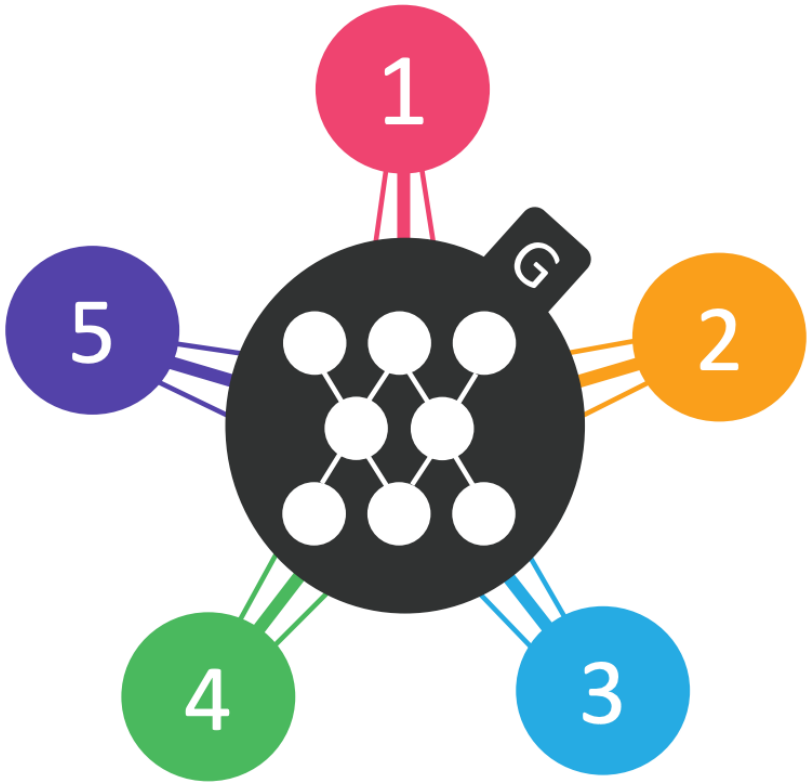
Zhu, et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks", ICCV 2017

Adversary as a Loss Function

- **Example: Muti-domain Image Translation (StarGAN)**
  - **Question:** What if there are multiple domains?

Pair-wise Translation



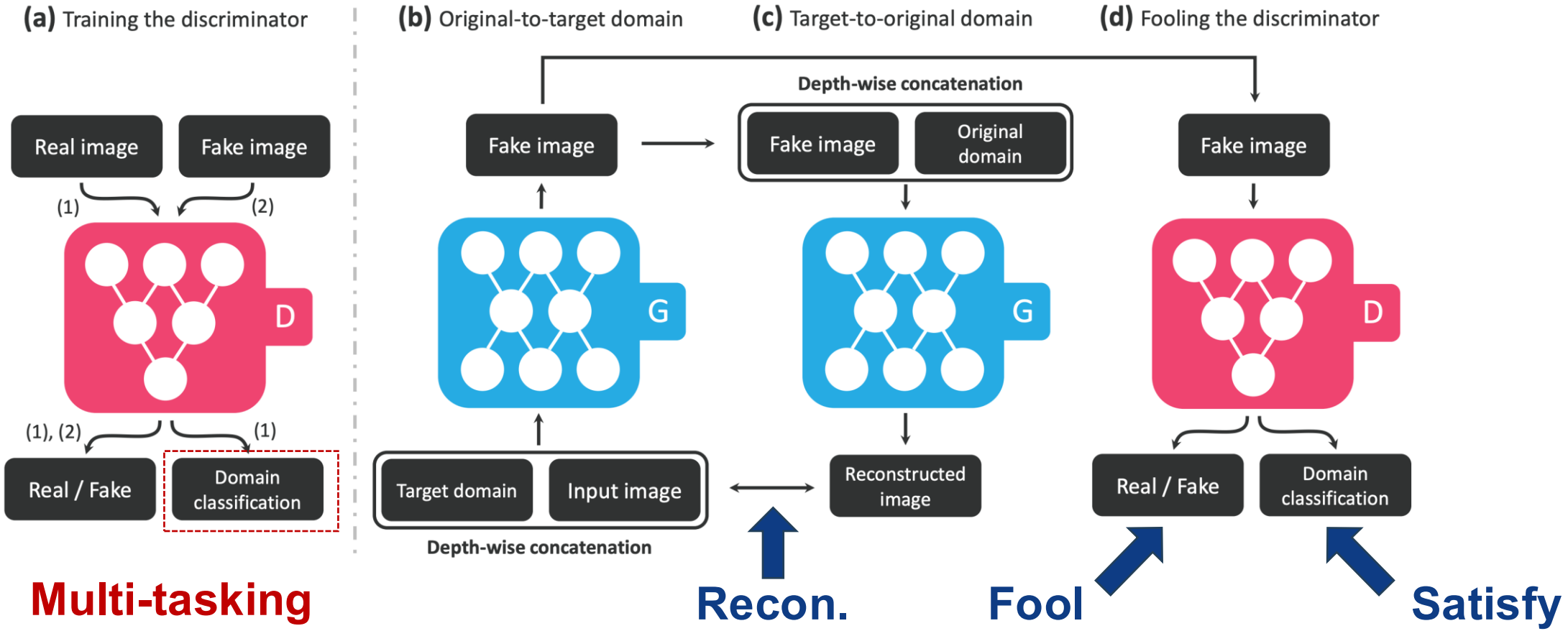


Translation via an Unified Model

StarGAN: Choi et al., "StarGAN: Unified Generative Adversarial Networks for Multi-Domain Image-to-Image Translation," CVPR 2018.

Adversary as a Loss Function

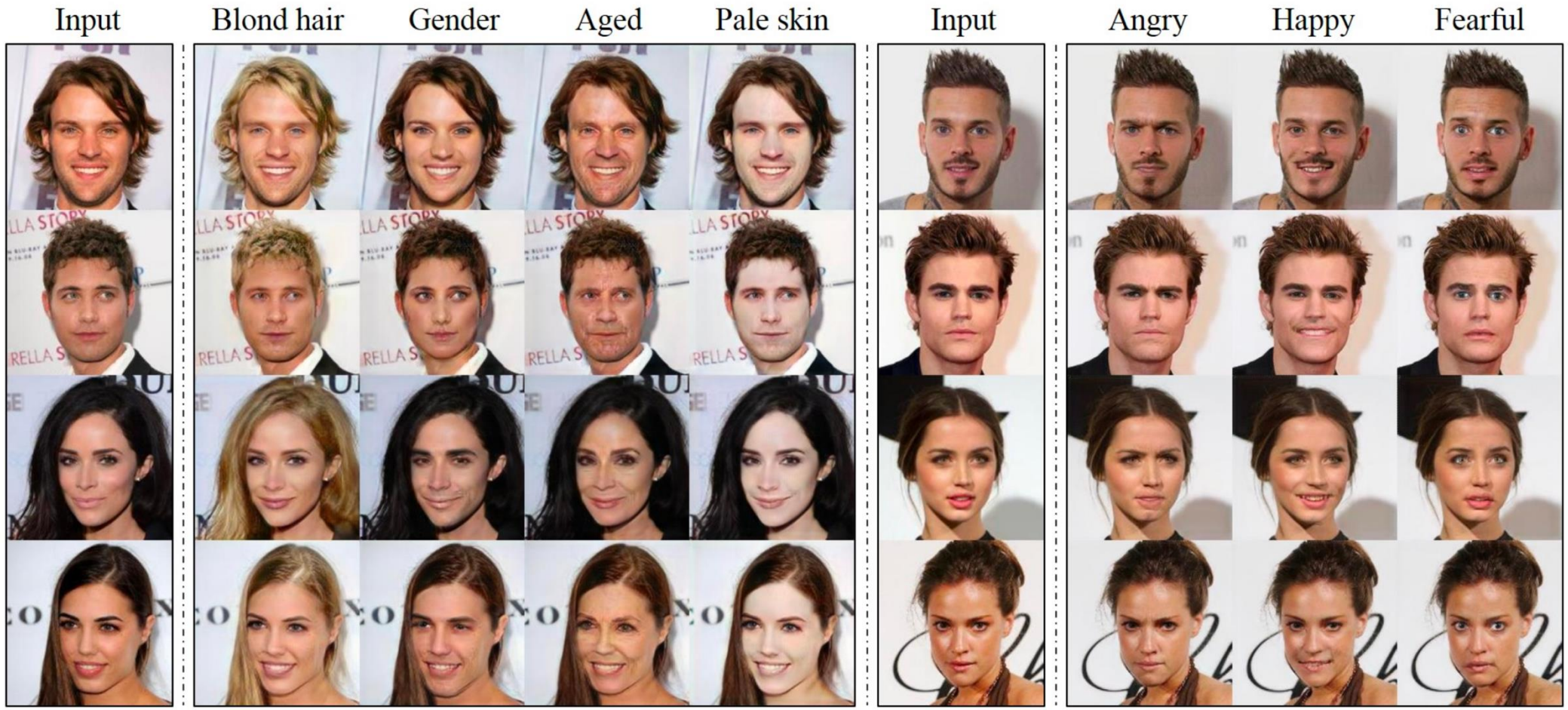
- **Example: Muti-domain Image Translation (StarGAN)**
  - **Question:** What if there are multiple domains?



StarGAN: Choi et al., "StarGAN: Unified Generative Adversarial Networks for Multi-Domain Image-to-Image Translation," CVPR 2018.

Adversary as a Loss Function

- **Example: Muti-domain Image Translation (StarGAN)**



StarGAN: Choi et al., "StarGAN: Unified Generative Adversarial Networks for Multi-Domain Image-to-Image Translation," CVPR 2018.

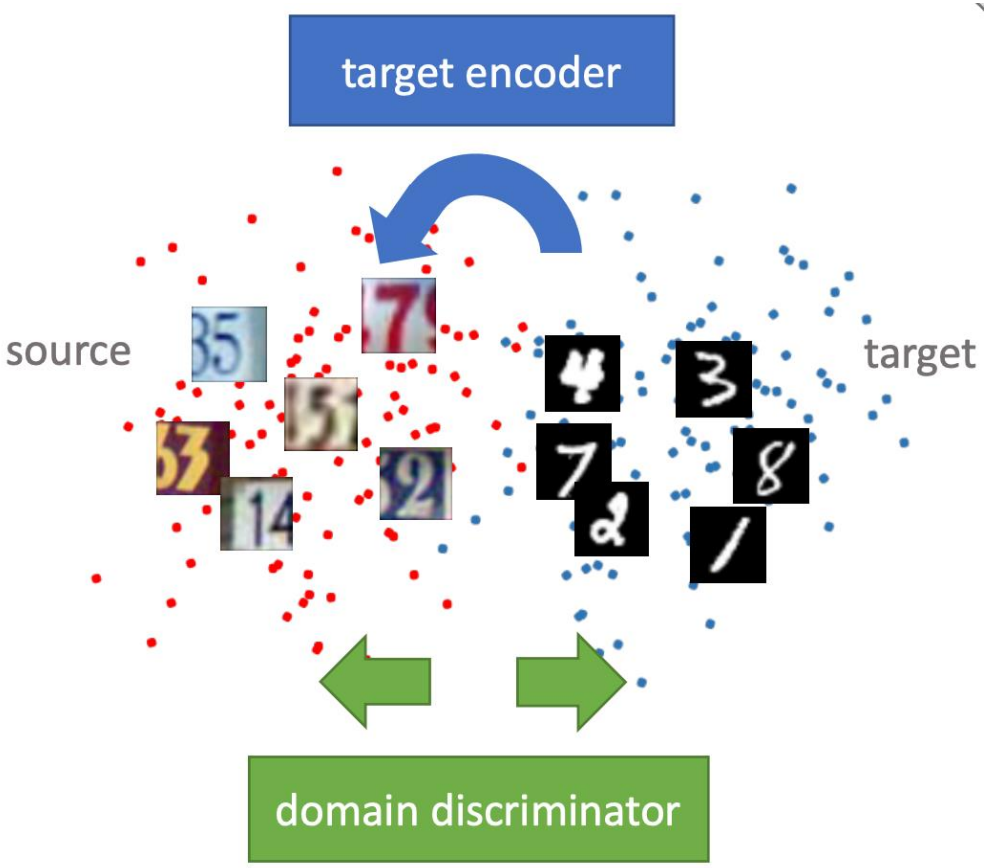
Adversary as a Loss Function

- **Example: Domain Adaptation (ADDA)**
  - **Question:** How to **generalize** the effect of **"aligning two distributions"**?

Source Domain

p_S

Labeled Data



Target Domain

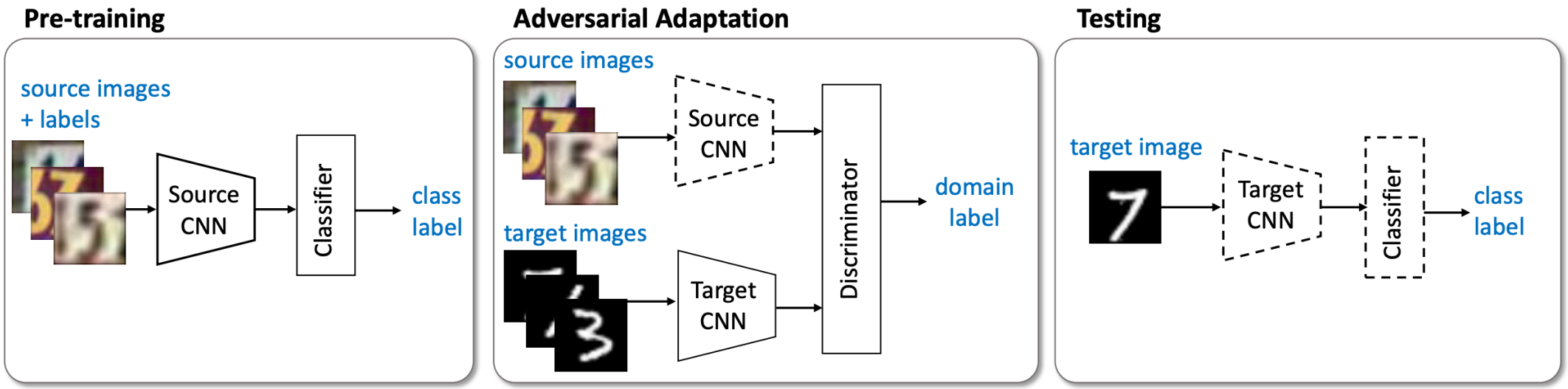
p_T

Unlabeled Data

Tzeng et al., "Adversarial Discriminative Domain Adaptation," CVPR 2017.

Adversary as a Loss Function

- **Example: Domain Adaptation (ADDA)**
  - **Question:** How to **generalize** the effect of **"aligning two distributions"**?



Train models with labeled source data

Align the feature space of p_S and p_T by **fooling the discriminator**

Re-use the trained model on target data

Tzeng et al., "Adversarial Discriminative Domain Adaptation," CVPR 2017.

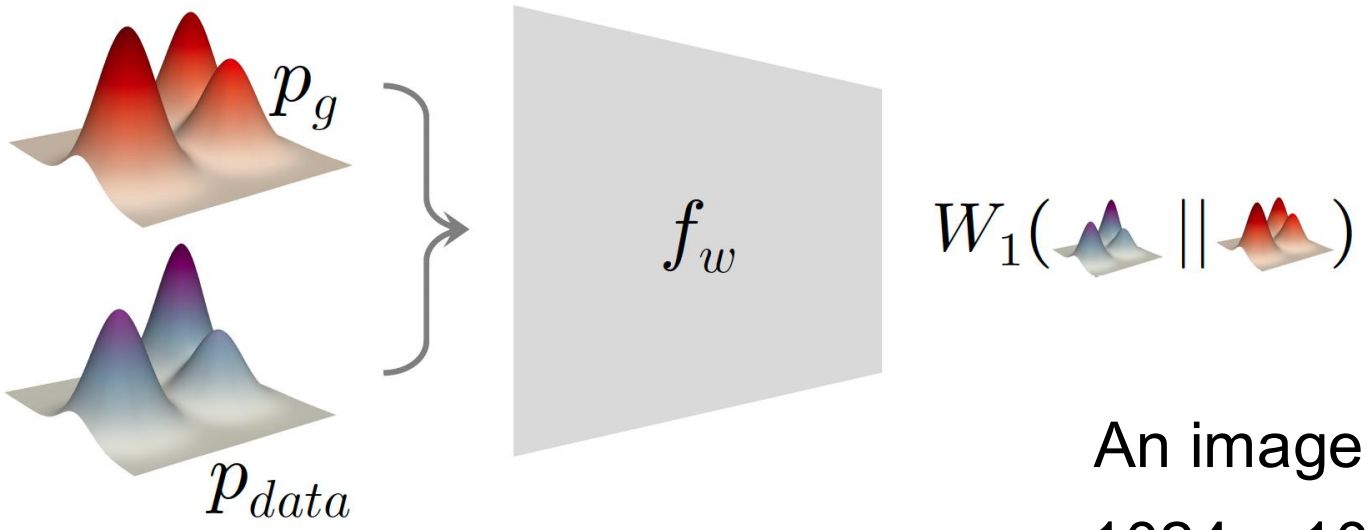


Lecture Outline

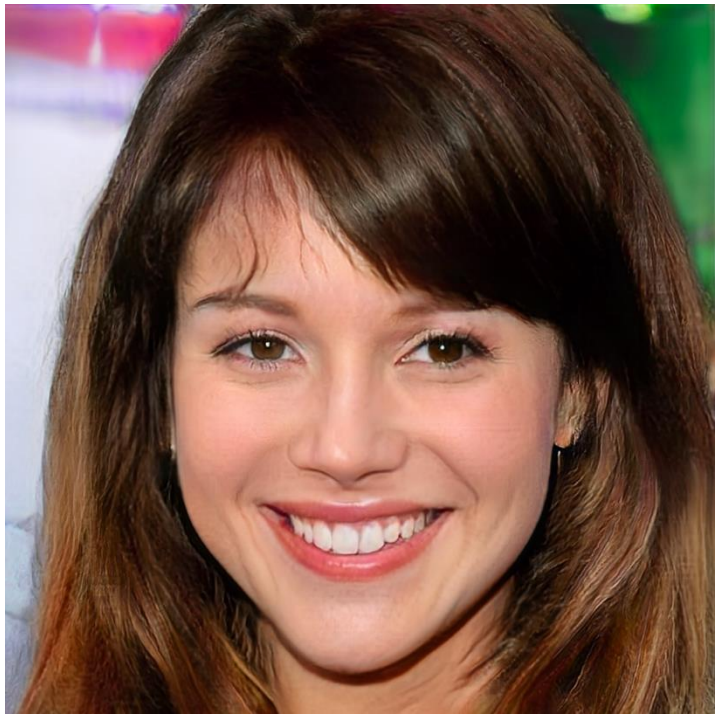
- **Wasserstein GAN (WGAN)**
- **Adversary as a Loss Function**
- **“Scaling” of GAN Models**

“Scaling” of GAN Models

- **Why GAN Scaling Matters?**
  - ➢ W-GAN provides an advanced objective which stabilizing training
    - ✓ **Question:** To scaling to larger images, can we **simple increase the output resolution** of the generator?



An image of 1024 x 1024

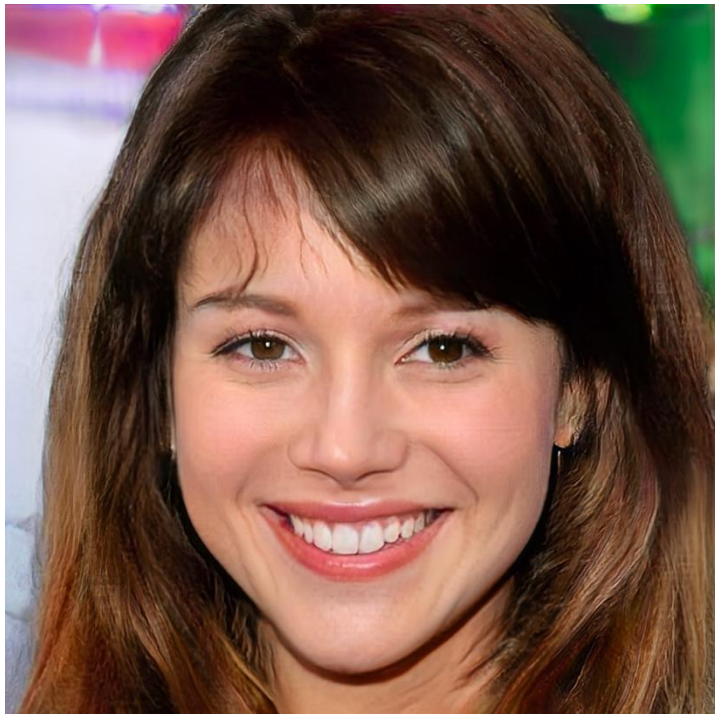


“Scaling” of GAN Models

- **Why GAN Scaling Matters?**
  - ➢ W-GAN provides an advanced objective which stabilizing training
    - ✓ **Question:** To scaling to larger images, can we **simple increase the output resolution** of the generator?
  - ➢ **Practical Difficulties**
    - ✓ **Optimization:**

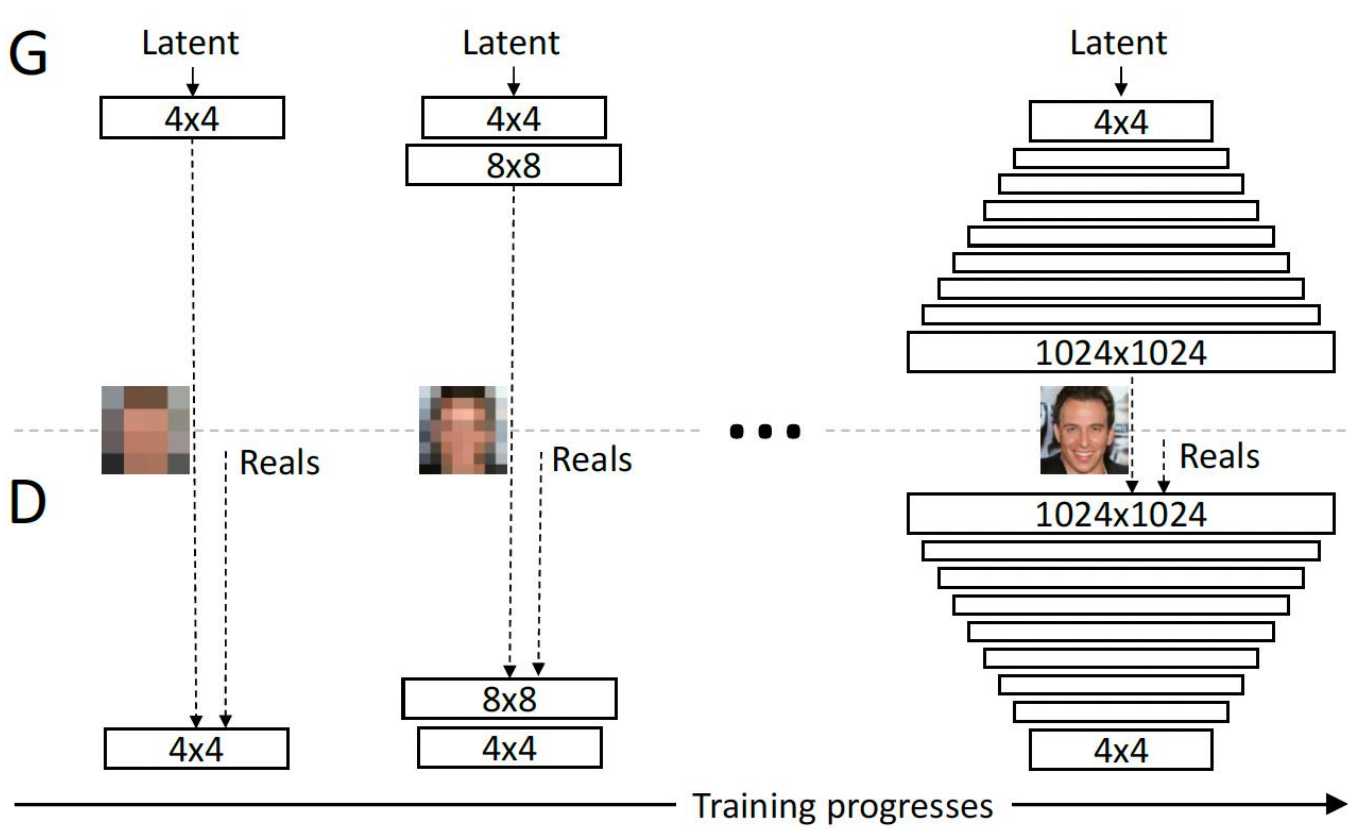
      D can easily overpower G
    - ✓ **Multi-scale Synthesis:**

      Local texture *vs.* Global Structure
    - ✓ **Latent Controllability**
    - ✓ **…**



“Scaling” of GAN Models

- **For Optimization: Progressive Growing of GANs (PG-GAN)**
  - **Motivation:** Increasing the generation resolution **gradually**

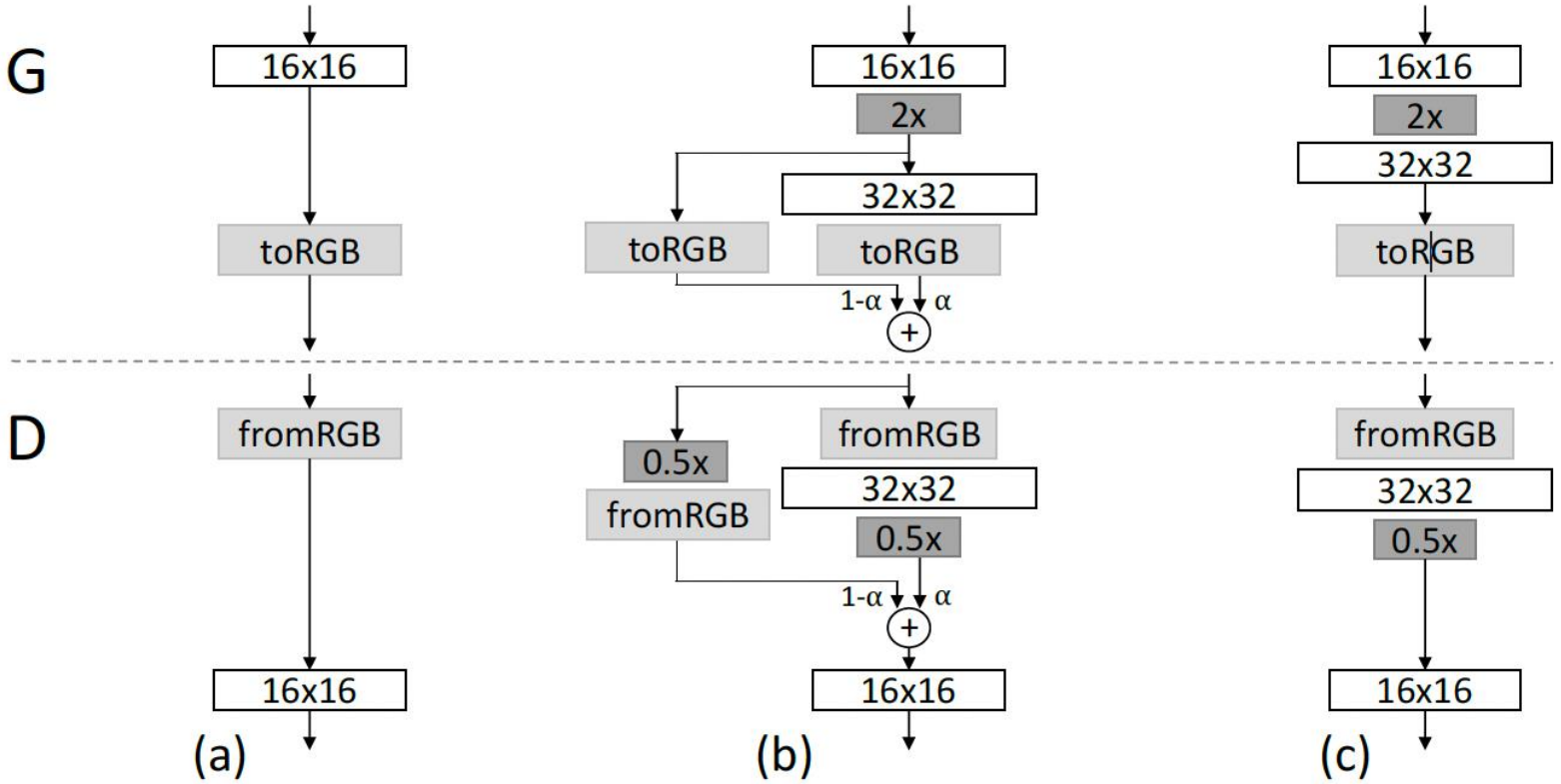




Karras et al., “Progressive Growing of GANs for Improved Quality, Stability, and Variation,” ICLR 2018

“Scaling” of GAN Models

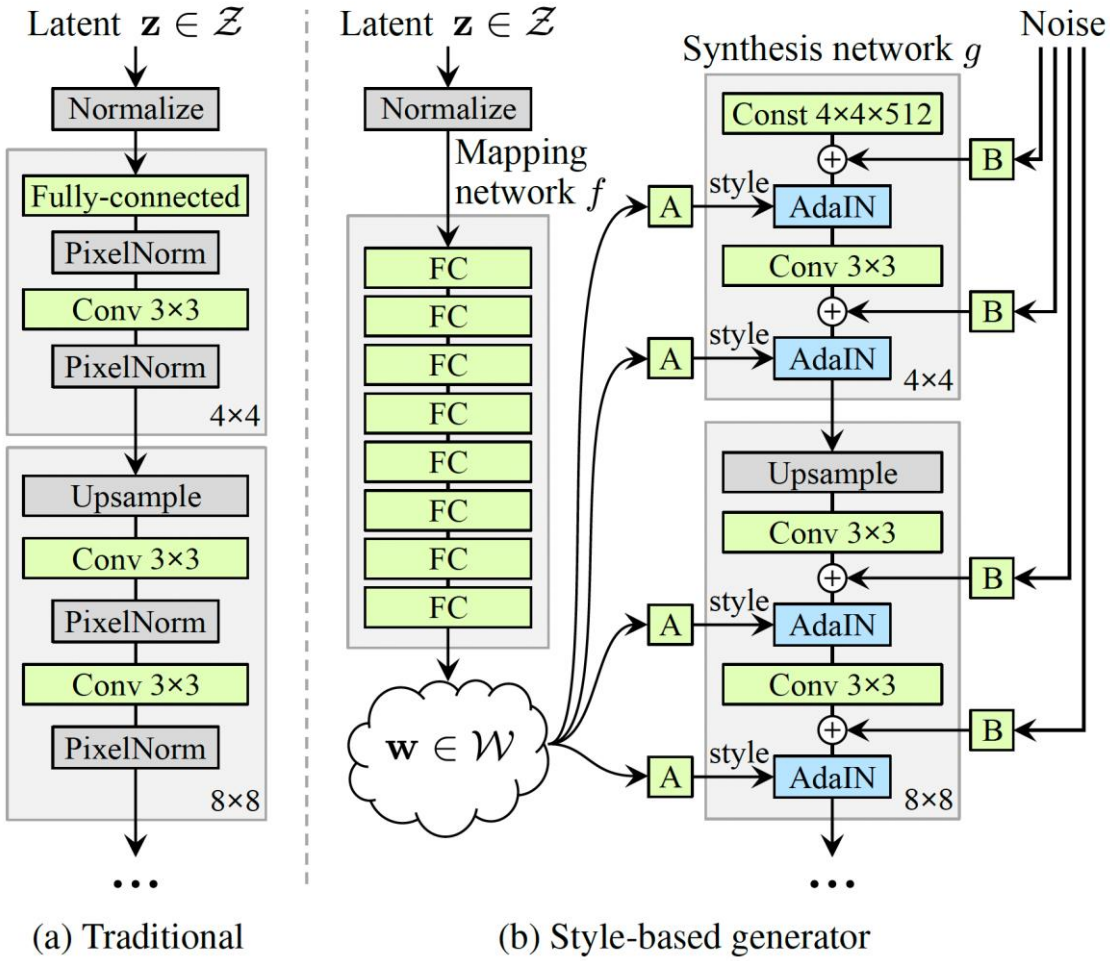
- **For Optimization: Progressive Growing of GANs (PG-GAN)**
  - **Motivation:** Increasing the generation resolution **gradually**



Karras et al., “Progressive Growing of GANs for Improved Quality, Stability, and Variation,” ICLR 2018

“Scaling” of GAN Models

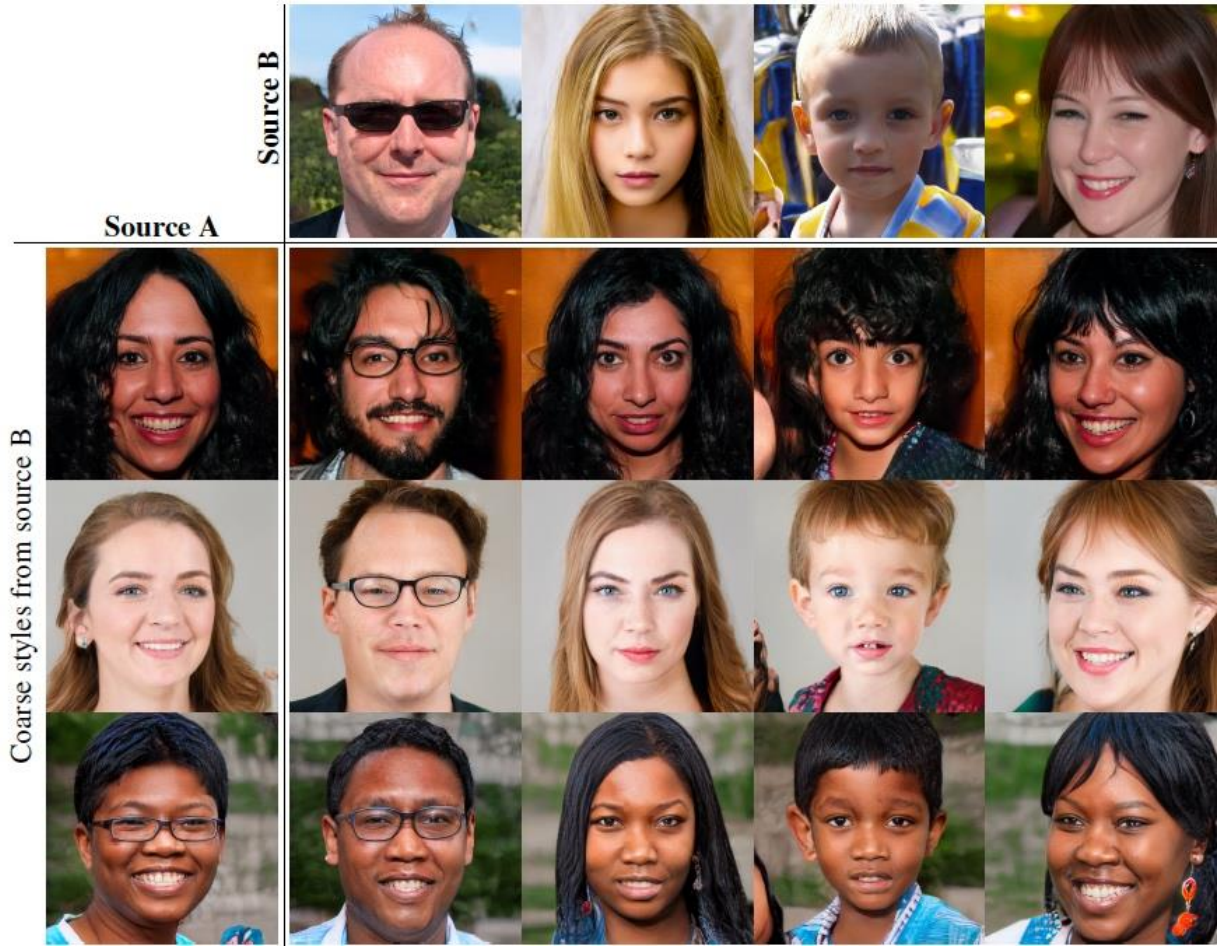
- **For Multi-Scale Synthesis and Controllability: StyleGAN**
  - **Motivation:** Design a novel architecture of generator
    - ✓ **Separately modeling** high-level attributes and stochastic variation
    - ✓ Latent input via **layer-wise injection / modulation**
    - ✓ Learn in **autonomous** and **unsupervised** manner

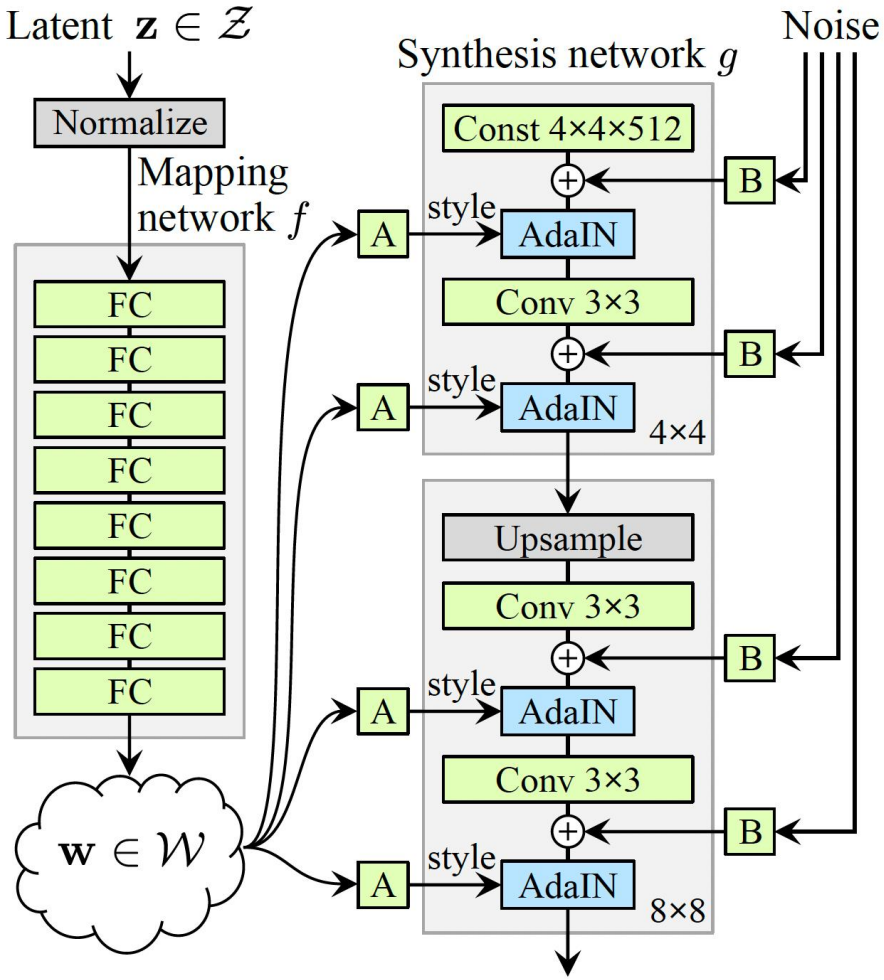


StyleGAN: Karras, Laine, and Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” CVPR 2019

“Scaling” of GAN Models

- **For Multi-Scale Synthesis and Controllability: StyleGAN**

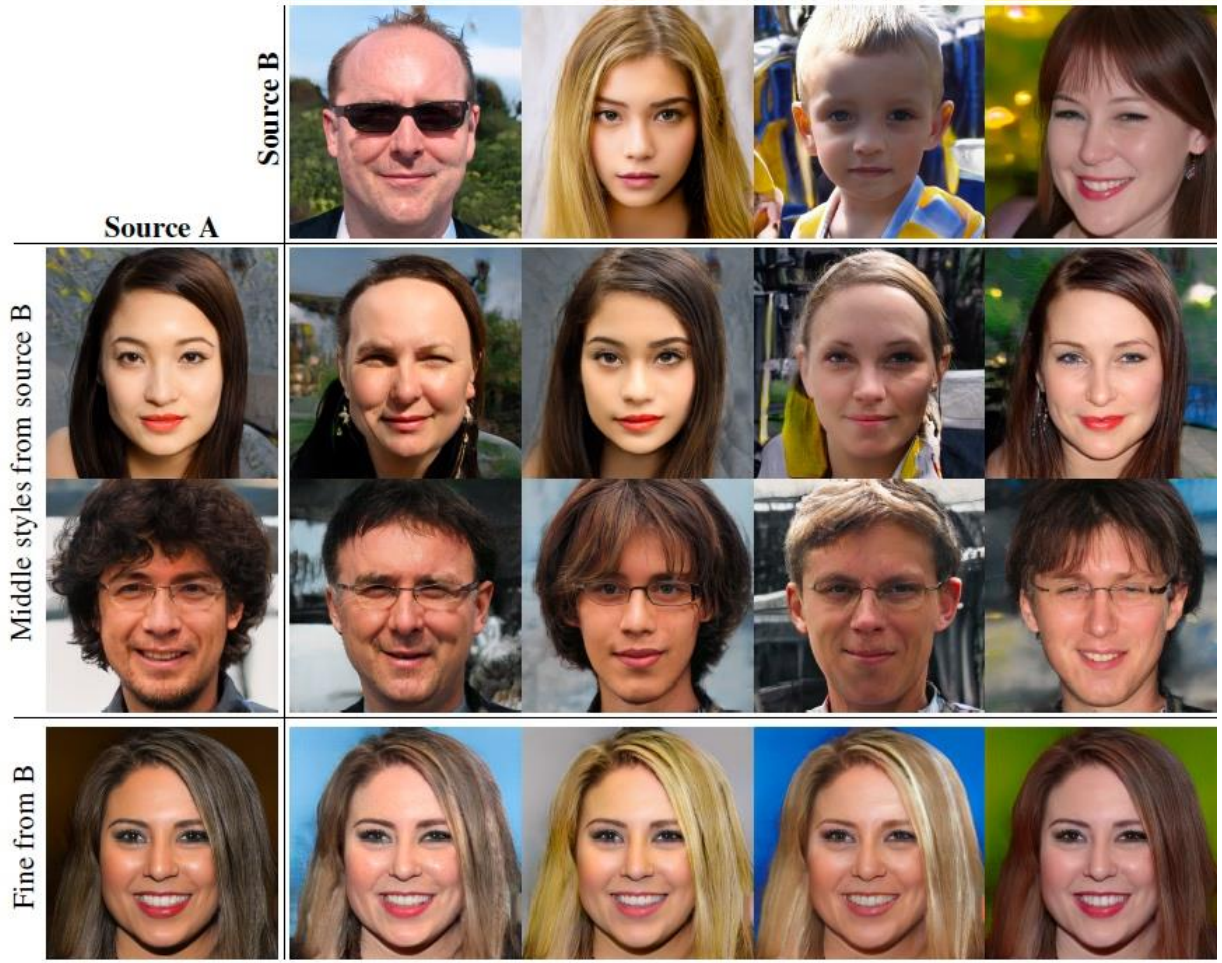


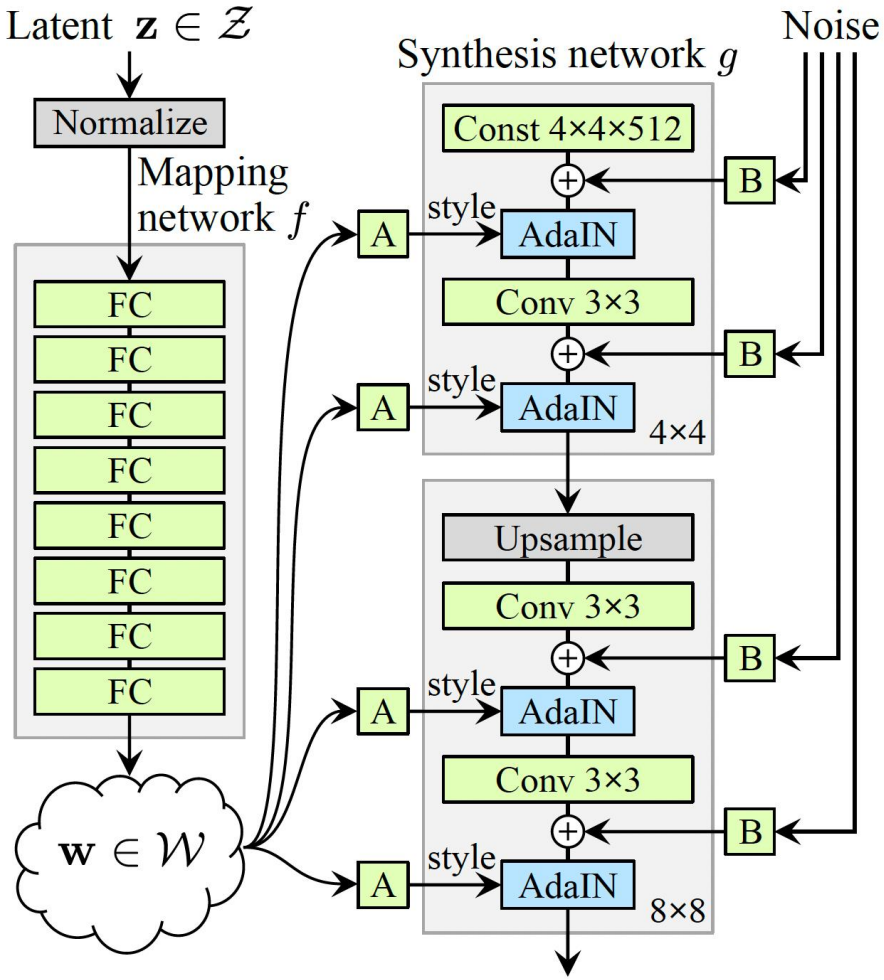


StyleGAN: Karras, Laine, and Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” CVPR 2019

“Scaling” of GAN Models

- **For Multi-Scale Synthesis and Controllability: StyleGAN**

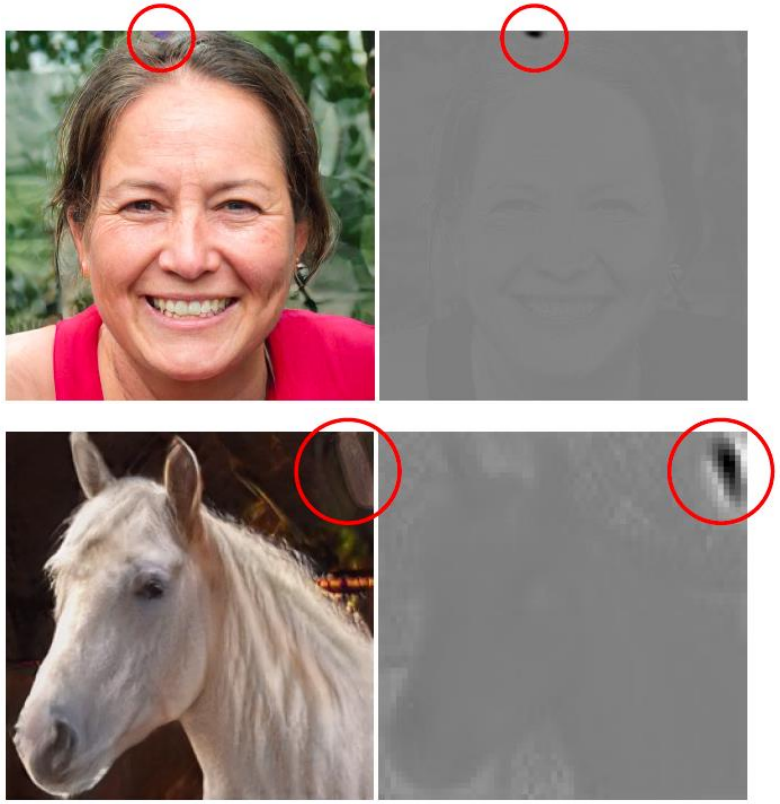




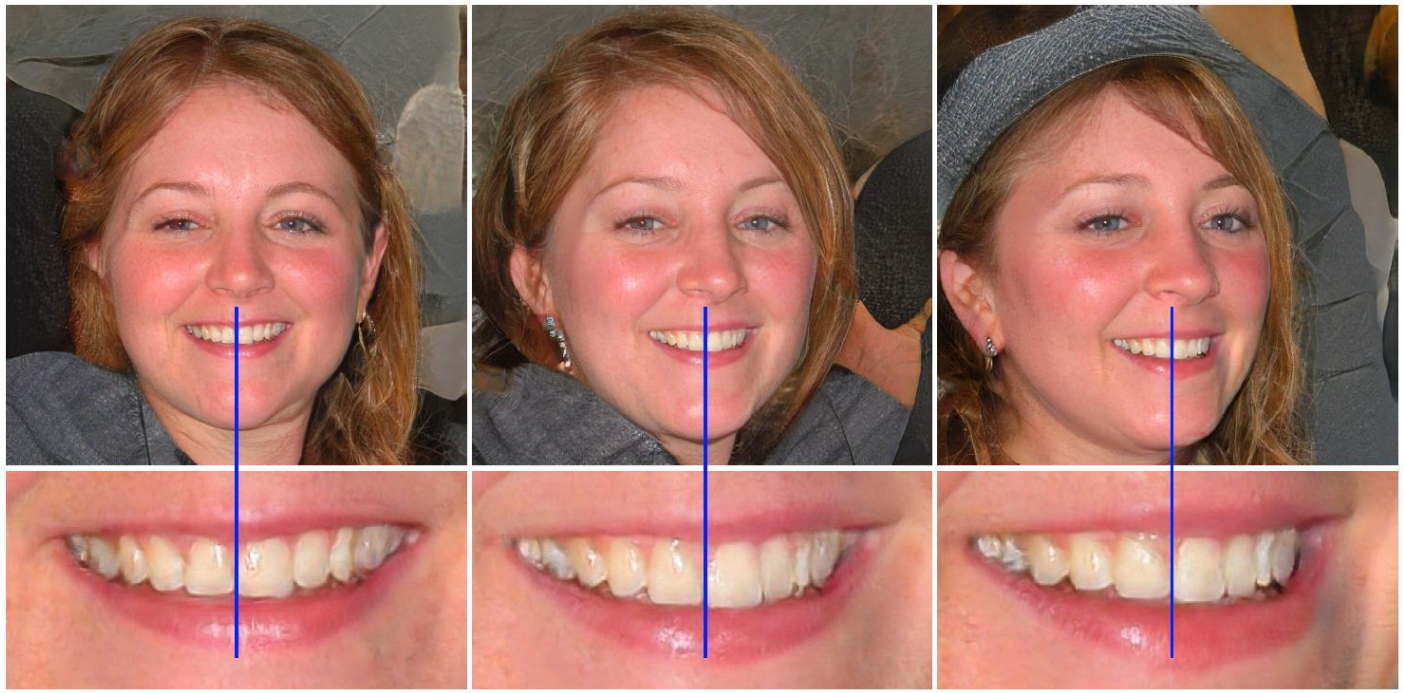
StyleGAN: Karras, Laine, and Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” CVPR 2019

“Scaling” of GAN Models

- **Engineering on StyleGAN: StyleGAN2**
  - ➢ Systematically improve the formulation and architecture of StyleGAN



Water Droplet-like Artifacts

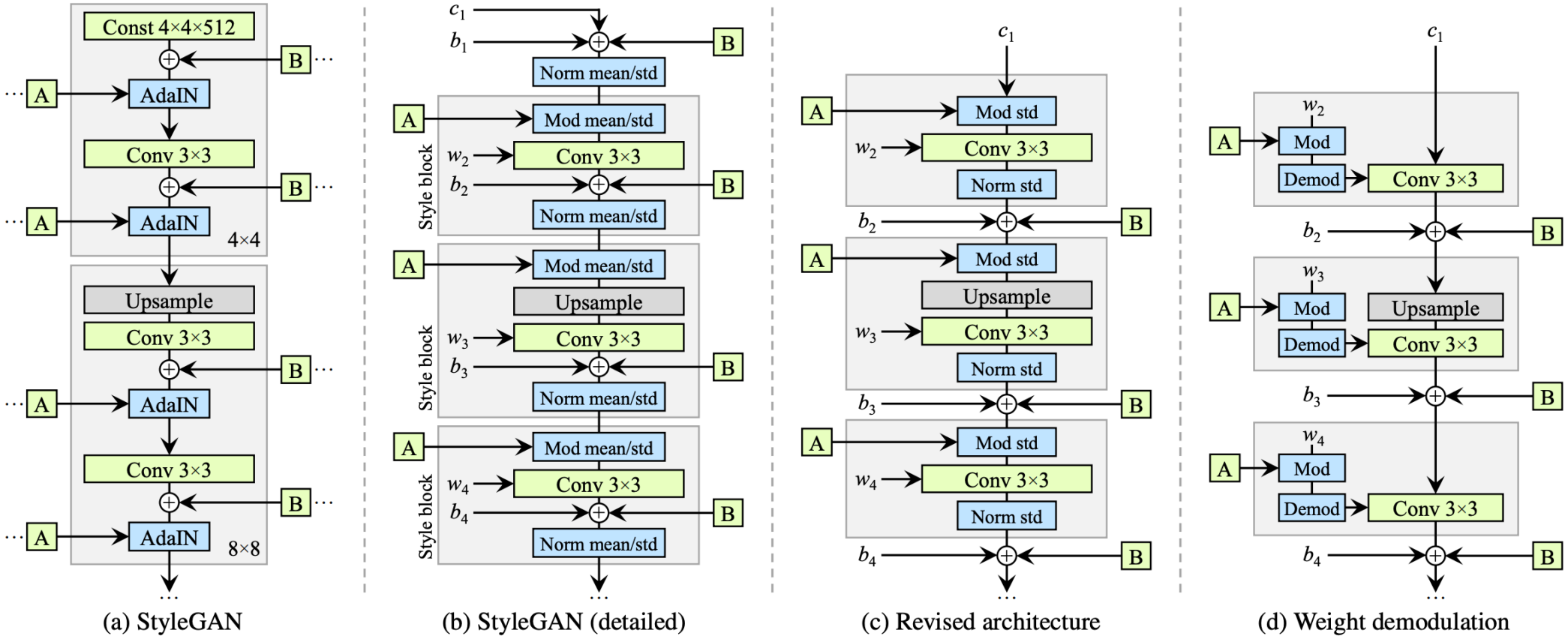


Unrealistic Perspective (teeth vs. pose)

StyleGAN2: Karras et al., “Analyzing and Improving the Image Quality of StyleGAN,” CVPR 2020.

“Scaling” of GAN Models

- **Engineering on StyleGAN: StyleGAN2**
  - ➢ Systematically improve the formulation and architecture of StyleGAN



StyleGAN2: Karras et al., “Analyzing and Improving the Image Quality of StyleGAN,” CVPR 2020.



“Scaling” of GAN Models

- **Further Reading: StyleGAN3 and Modern GANs**

- Karras et al., “Alias-Free Generative Adversarial Networks,” **NeurIPS 2021 (StyleGAN3)**
- Sauer et al., “Projected GANs Converge Faster,” **NeurIPS 2021**
- Gal et al., “StyleGAN-NADA: CLIP-Guided Domain Adaptation of Image Generators,” **SIGGRAPH 2022**
- Sauer et al., “StyleGAN-XL: Scaling StyleGAN to Large Diverse Datasets,” **SIGGRAPH 2022**
- Chan et al., “Efficient Geometry-aware 3D Generative Adversarial Networks,” **CVPR 2022**
- Gao et al., “GET3D: A Generative Model of High Quality 3D Textured Shapes Learned from Images,” **NeurIPS 2022**
- Sauer et al., “StyleGAN-T: Unlocking the Power of GANs for Fast Large-Scale Text-to-Image Synthesis,” **ICML 2023**
- Kang et al., “Scaling Up GANs for Text-to-Image Synthesis,” **CVPR 2023**
- Wang et al., “Diffusion-GAN: Training GANs with Diffusion,” **ICLR 2023**
- Huang et al., “The GAN is dead; long live the GAN! A Modern GAN Baseline,” **NeurIPS 2024**



Lecture Outline

- **Wasserstein GAN (WGAN)**
- **Adversary as a Loss Function**
- **“Scaling” of GAN Models**

Supplementary Material

- **Stanford CS 236 Lecture 10**
  - https://www.youtube.com/watch?v=M3Fkvu78ZXc&list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8&index=10
- **Cornell CS 6785 Lecture 10**
  - https://www.youtube.com/watch?v=MI15crPldBk&list=PL2UML_KCiC0UPzjW9BjO-IW6dqliu9O4B&index=10
- **Deep Generative Models**
  - Chapter 8 Generative Adversarial Networks